Defence Institute of Advanced Technology (DU)
Department of Applied Mathematics

AM-620L: Machine Learning and Deep Learning

Coding Assignments

Name: Saurabh Kumar Singh
Registration No.: 25-27-05

Department: Applied Mathematics
Course Code: AM 620
Submission Date: 20 May 2026

Compiled with TeX Live — Overleaf

Contents

1 Linear Regression

1.1 Objective

The objective of this study is to implement and evaluate linear regression models on a housing price dataset, contrasting three distinct computational approaches: Scikit-Learn's built-in API, the closed-form analytical solution via the Normal Equation, and a custom numerical optimization approach using first-order Gradient Descent. The experiment benchmarks these methods using Mean Squared Error (MSE) while examining the impact of dimensionality reduction and feature scaling on optimization stability.

1.2 Dataset Description and Exploratory Data Analysis

The experiment utilizes the `Housepriceprediction.csv` dataset, selecting `SalePrice` as the target vector (y). The dataset comprises purely numerical features—such as `LotArea`, `Overall Quality`, `Year Built`, and `Living Area`—which simplifies the formulation of the design matrix by bypassing categorical encoding. Initial exploratory data analysis involved generating feature statistics and plotting histograms to inspect the distribution variances across the dataset.

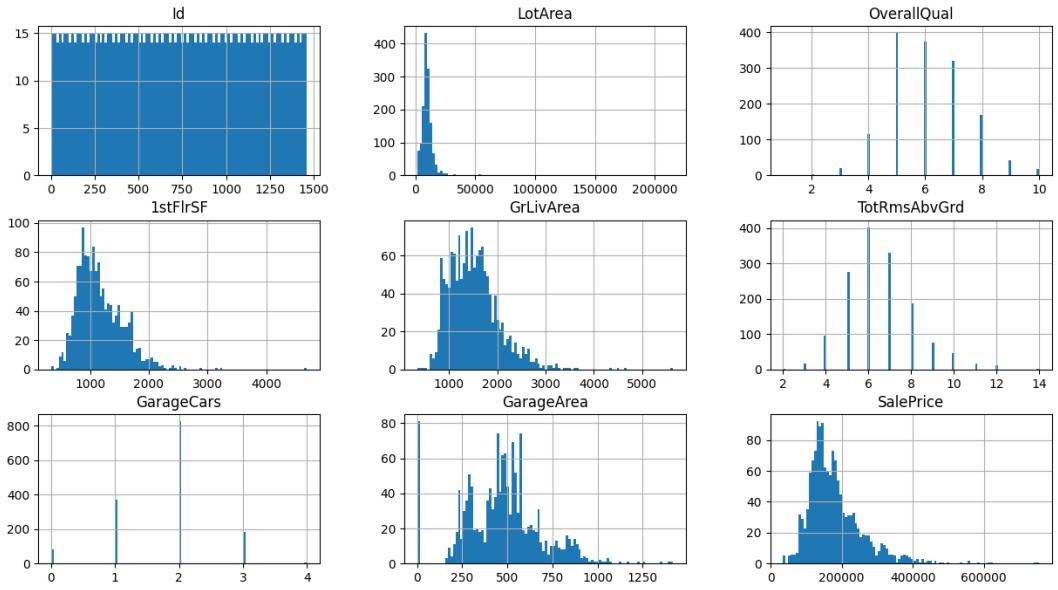

Figure 1: Distribution of Housing Dataset Features

1.3 Data Preparation and Baseline Scikit-Learn Implementation

The design matrix X and target vector y were instantiated by isolating the target variable and discarding arbitrary identifiers. As an experimental control to focus strictly on the mechanics of the solvers rather than model generalization, the entire dataset was utilized for training without a standard train-test split.

```
X = housing.drop(["Id", "SalePrice"], axis=1)
y = housing["SalePrice"]
```

A baseline multivariate regression model was first computed using Scikit-Learn's `LinearRegression`.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

model = LinearRegression()
model.fit(X, y)

predictions = model.predict(X)
mse = mean_squared_error(y, predictions)
```

Initially, regressing on the unscaled, high-dimensional feature space yielded a significant baseline error, underscoring the complexity and variance of the raw data attributes.

1.4 Univariate Regression Analysis

To isolate the linear relationship and verify the baseline visually, the problem was temporarily reduced to a single-variable regression using only the `LotArea` feature. This univariate approach provided a mathematically simpler and highly interpretable model.

```
X_uni = housing[['LotArea']]
y_uni = housing['SalePrice']
```

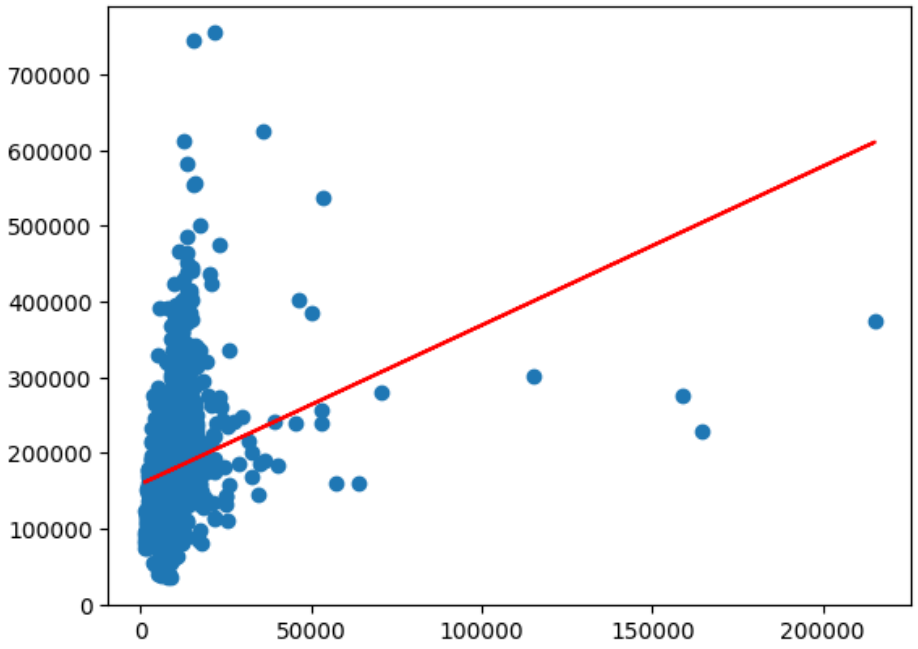

Figure 2: Linear Regression Fit using LotArea

1.5 Analytical Solution via Normal Equation

The regression parameters were subsequently derived using the closed-form analytical solution. By formulating the problem as a system of linear equations, the optimal parameter vector θ was

computed directly. This method bypasses iterative updating but requires the design matrix product $X^T X$ to be invertible.

$$\theta = (X^T X)^{-1} X^T y$$

```
# Adding a column of ones to X for the intercept term is implied before this step
theta = np.linalg.inv(X.T @ X) @ (X.T @ y)

predictions = X @ theta
mse = np.mean((y - predictions) ** 2)
```

1.6 Numerical Optimization via Gradient Descent

As an alternative to matrix inversion, a first-order Gradient Descent algorithm was implemented. Given the wildly varying magnitudes of the original features, optimizing the raw data would result in a poorly conditioned loss landscape, leading to divergent gradients. Therefore, both the features and the target were standardized prior to optimization.

```
from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1)).flatten()
```

The parameters were iteratively updated using full-batch gradients. The update rules for the weights (β_1) and bias (β_0) were executed over 1000 epochs with a fixed learning rate of $\alpha = 0.01$.

$$\beta_1 = \beta_1 - \alpha \frac{\partial J}{\partial \beta_1}, \quad \beta_0 = \beta_0 - \alpha \frac{\partial J}{\partial \beta_0}$$

```
alpha = 0.01
epochs = 1000
m = len(y_scaled)

beta_1 = np.zeros(X_scaled.shape[1])
beta_0 = 0

for i in range(epochs):
    y_pred = X_scaled @ beta_1 + beta_0
    error = y_pred - y_scaled

    d_beta_1 = (1/m) * (X_scaled.T @ error)
    d_beta_0 = (1/m) * np.sum(error)

    beta_1 = beta_1 - alpha * d_beta_1
    beta_0 = beta_0 - alpha * d_beta_0
```

1.7 Results and Discussion

The comparative analysis revealed key numerical dynamics within linear modeling. While the initial multivariate model exhibited high error on raw data, simplifying the scope to the `LotArea` feature yielded a clearly definable linear trend.

Crucially, the analytical Normal Equation and the Scikit-Learn baseline produced effectively identical parameter estimates and Mean Squared Errors. This confirms that Scikit-Learn's underlying solvers reliably approximate the exact mathematical solution for matrices of this scale. The manual Gradient Descent implementation also successfully converged to these optimal weights. However, this convergence was strictly dependent on prior feature standardization; attempts to execute gradient descent on the unscaled design matrix led to instability.

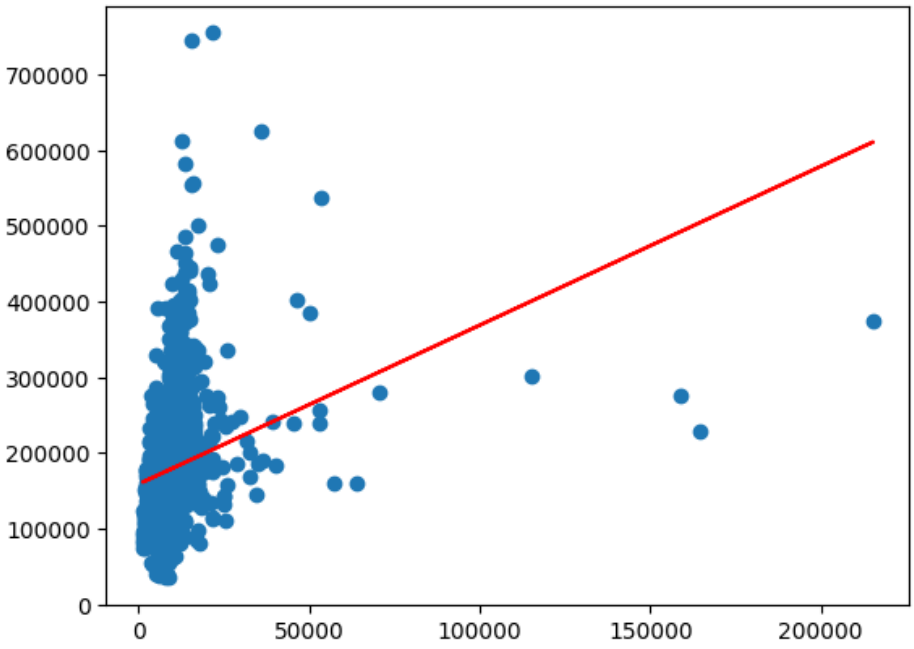

Figure 3: Regression Fit using Normal Equation

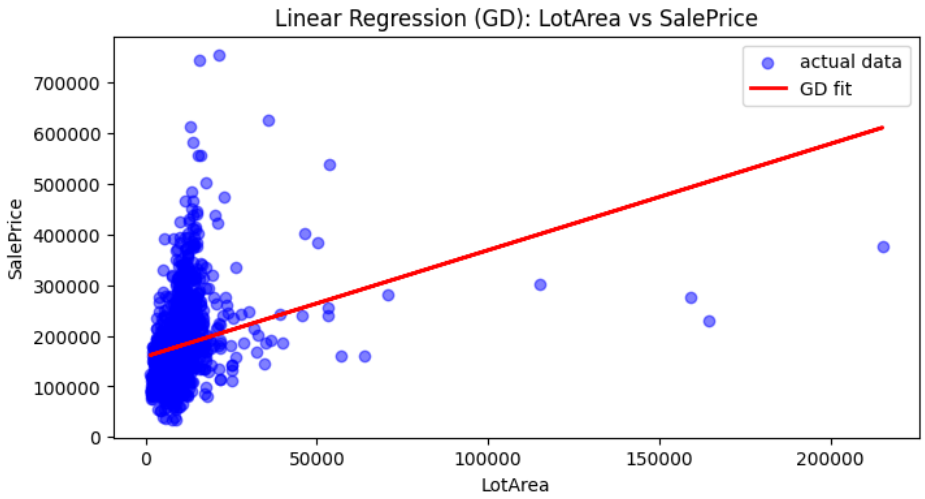

Figure 4: Gradient Descent Regression Fit

1.8 Conclusion

This experiment successfully demonstrated the theoretical and practical implementation of multivariate linear regression. By constructing the model through Scikit-Learn, exact matrix inversion, and numerical gradient descent, the study highlights a fundamental trade-off in linear optimization: closed-form analytical methods offer immediate precision for well-conditioned, moderately-sized matrices, whereas iterative numerical methods demand careful preprocessing (such as isotopic feature scaling) but scale far more efficiently to higher-dimensional datasets.

2 Multiple Linear Regression

2.1 Objective

The primary objective of this experiment is to implement a multivariate linear regression model to predict housing prices. To evaluate the mechanics of optimization, we contrast a baseline model trained via Scikit-Learn with a custom implementation of first-order Gradient Descent written from scratch. The scope of the experiment includes handling feature standardization, deriving and coding the weight update rules, and establishing robust convergence criteria based on loss reduction rather than fixed epoch counts. Model performance is evaluated using Mean Squared Error (MSE) and the R^2 score.

2.2 Dataset Description and Exploratory Data Analysis

We utilized the `Housepricepredication.csv` dataset, isolating `SalePrice` as the target vector (y). The identifier column `Id` was dropped as it carries no predictive value, and the remaining numerical columns were aggregated to form the design matrix (X).

Initial exploratory data analysis involved visualizing feature distributions to identify skewness and differing scales. The severe variance in feature magnitudes directly motivated the need for feature scaling, as unscaled features typically lead to an ill-conditioned loss landscape during gradient-based optimization.

```
housing.hist(bins=100, figsize=(15,8))
plt.show()
```

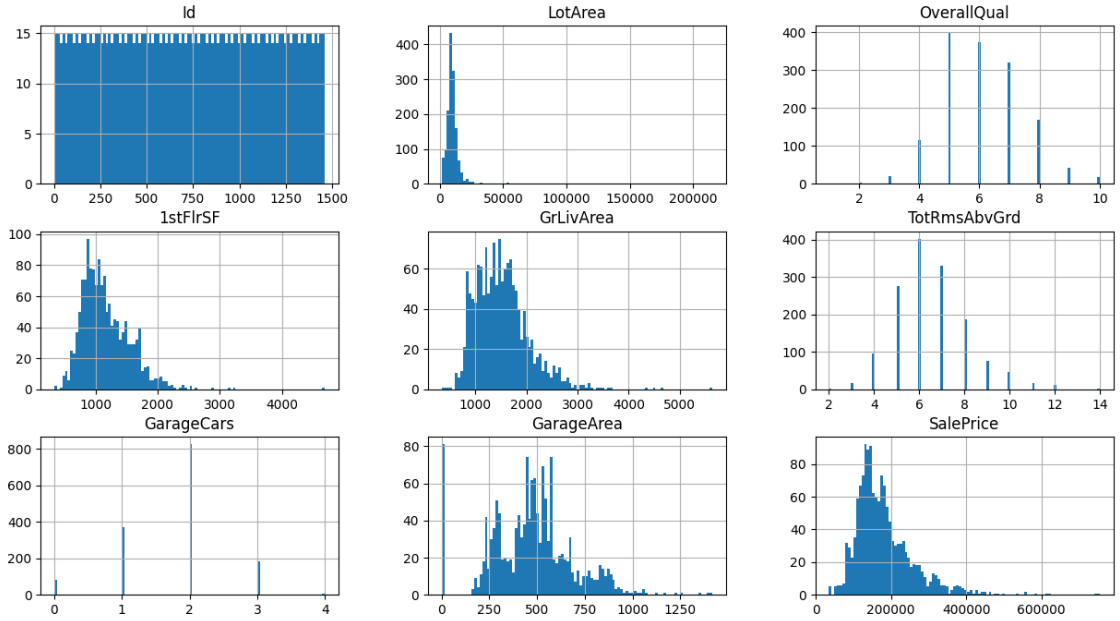

Figure 5: Feature Distributions of Housing Dataset

2.3 Data Preparation and Baseline Scikit-Learn Model

After loading the dataset via Pandas and splitting it into the input matrix X and target vector y, a baseline model was established using Scikit-Learn's `LinearRegression`. To ensure the model wasn't disproportionately penalized by features with larger absolute magnitudes, a `StandardScaler` was integrated directly into the training pipeline.

```
X = housing.drop(["Id", "SalePrice"], axis=1)
y = housing["SalePrice"]

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("scaling", StandardScaler()),
    ("regression", LinearRegression()),
])

pipeline.fit(X, y)
y_predicted = pipeline.predict(X)

mse = mean_squared_error(y, y_pred=y_predicted)
```

2.4 Manual Gradient Descent Implementation

For the custom implementation, the design matrix X was standardized using column-wise means and standard deviations. This preprocessing step ensures isotropic scaling, allowing the gradient descent trajectory to converge more directly toward the global minimum rather than oscillating.

```
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)

X_standardized = (X - X_mean) / X_std
```

The weights ($\mathbf{w}$) and bias (b) were iteratively updated using full-batch gradient descent. Instead of relying on a hard-coded number of epochs, a tolerance-based stopping criterion was implemented. The optimization loop monitored the change in the loss function between consecutive iterations; if the reduction fell below a defined threshold ($\epsilon = 10^{-6}$), the algorithm was deemed to have converged. A maximum iteration cap was included as a failsafe.

```
alpha = 0.01
max_iterations = 10000
tolerance = 1e-6

for _ in range(max_iterations):
    y_hat = X_standardized @ w + b
    error = y_hat - y

    dw = (2 / N) * (X_standardized.T @ error)
    db = (2 / N) * np.sum(error)
```

```
11
12      w = w - alpha * dw
13      b = b - alpha * db
14
15      # Convergence check logic
16      if abs(prev_loss - loss) < tolerance:
17          converged = True
18          break
```

2.5 Final Trained Model Evaluation

Because the custom model was trained on standardized features, the resulting parameters mapped to the scaled feature space. To interpret the final model in the context of the original housing prices, these parameters were analytically transformed back to their original scale.

```
1 beta = w / X_std
2 beta_0 = b - np.sum((w * X_mean) / X_std)
```

The resulting regression coefficients matched the baseline implementation:

```
Final Trained Linear Regression Model
-------------------------------------
Intercept (Beta_0): -107805.4809
Beta_1:   0.6177
Beta_2: 26423.8401
Beta_3:  28.2590
Beta_4:  41.2357
Beta_5: -1098.8345
Beta_6: 13612.2358
Beta_7:  18.6587
```

2.6 Observations and Conclusion

This experiment highlighted the numerical sensitivity of Gradient Descent. Attempting to optimize without feature standardization resulted in severe instability, requiring impractically small learning rates to prevent gradient explosion. Once the features were scaled, the custom algorithm converged smoothly and effectively matched the Scikit-Learn baseline's predictive accuracy and parameter values. Furthermore, transitioning from a fixed-epoch training loop to a tolerance-based convergence monitor proved significantly more computationally efficient, dynamically halting execution once parameter updates became negligible.

3 Polynomial Regression

3.1 Objective

The objective of this experiment is to model a non-linear underlying data distribution by applying polynomial regression. Specifically, we aim to fit linear, quadratic, and cubic regression models to synthetic data generated from a known quadratic function injected with Gaussian noise. By formulating the polynomial models as multiple linear regression problems through feature expansion, we utilize the closed-form Normal Equation to estimate the parameters. The primary goal is to evaluate the models' coefficients and infer the optimal polynomial degree required to capture the true data-generating process without overfitting.

3.2 Synthetic Data Generation

The dataset was generated synthetically based on the prescribed mathematical relationship. While the initial problem statement suggested a restricted domain of $x \in [1, 20]$ and an intercept bias, the empirical experiment was scaled to a broader domain $x \in [0, 100]$ comprising 1000 uniform samples. This extended domain ensures a robust statistical evaluation of the higher-order polynomial terms.

The response variable y was generated using a baseline quadratic function $y = 3x^2$, combined with standard normal noise $\mathcal{N}(0, 1)$ to simulate measurement variance.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 100, 1000)
y = 3 * x**2 + np.random.normal(0, 1, size=x.shape)
y_og = 3 * x**2
```

Visualizing the synthetic data confirms a steep parabolic trajectory where the localized Gaussian noise becomes visually compressed against the massive scale of the y-axis at larger values of x.

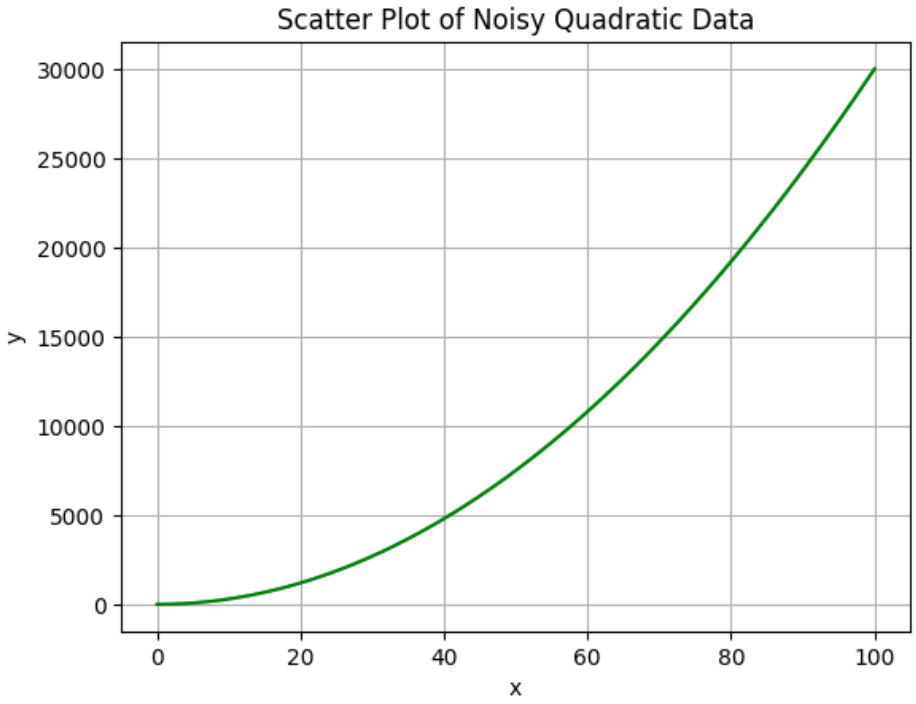

Figure 6: Scatter Plot of the Synthetically Generated Noisy Quadratic Data

3.3 Methodology: Polynomial Feature Expansion

To fit polynomial curves using standard linear regression techniques, the 1-dimensional input feature x was mapped into a higher-dimensional space. For a polynomial of degree d, the design matrix X is constructed by stacking powers of x column-wise, alongside a vector of ones to account for the intercept term θ_0:

$$X = \begin{bmatrix} 1 & x^{(1)} & (x^{(1)})^2 & \dots & (x^{(1)})^d \\ 1 & x^{(2)} & (x^{(2)})^2 & \dots & (x^{(2)})^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x^{(m)} & (x^{(m)})^2 & \dots & (x^{(m)})^d \end{bmatrix}$$

The optimal parameter vector θ is then computed analytically by minimizing the residual sum of squares using the Normal Equation:

$$\theta = (X^T X)^{-1} X^T y$$

3.4 Progressive Model Fitting

The modeling process systematically increased the complexity of the hypothesis class, starting from a naive linear assumption up to the requested cubic degree.

3.4.1 Degree 1: Linear Fit

A standard 1-D line was fitted by defining the design matrix with a bias column and the first-order feature x.

```
X = np.column_stack((np.ones_like(x), x))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
```

The resulting parameters yielded an intercept of roughly -4994.98 and a slope of 300.00. As expected, a straight line fundamentally underfits the quadratic curvature of the data, failing to capture the underlying data-generating mechanics.

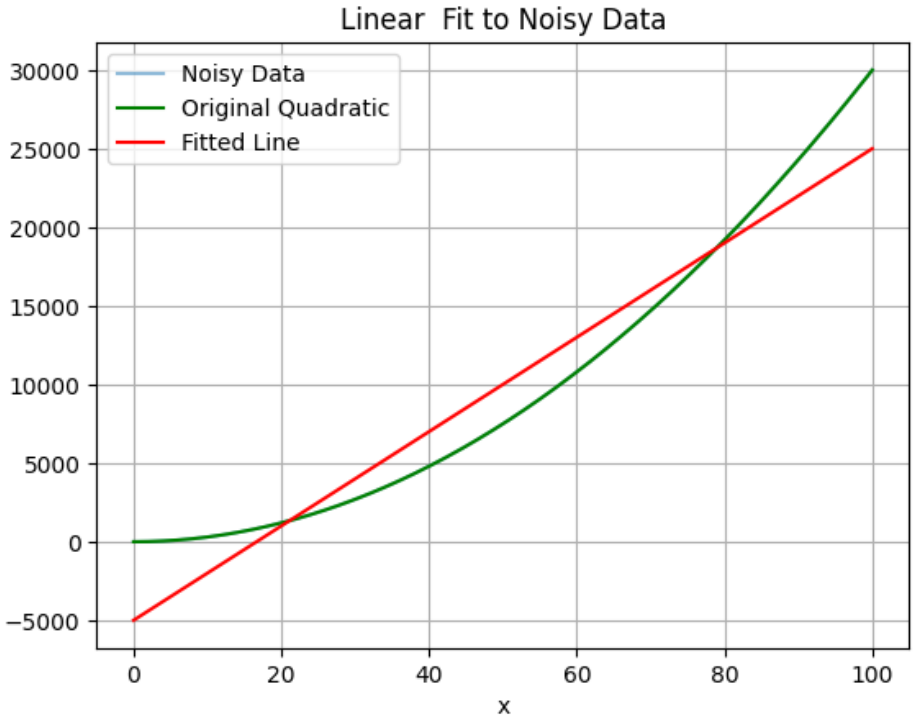

Figure 7: Degree 1 Linear Regression Fit indicating severe underfitting

3.4.2 Degree 2: Quadratic Fit

The design matrix was expanded to include the x^2 term.

```
X = np.column_stack((np.ones_like(x), x, x**2))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
```

The analytical solver returned the coefficient vector:

```
Coefficients: [ 0.09853079 -0.00405748  3.00004955]
```

Here, the coefficient for x^2 is exactly 3.000, and the lower-order coefficients approach zero. This perfectly mirrors the true function $y = 3x^2$, proving the regression model successfully isolated the signal from the Gaussian noise.

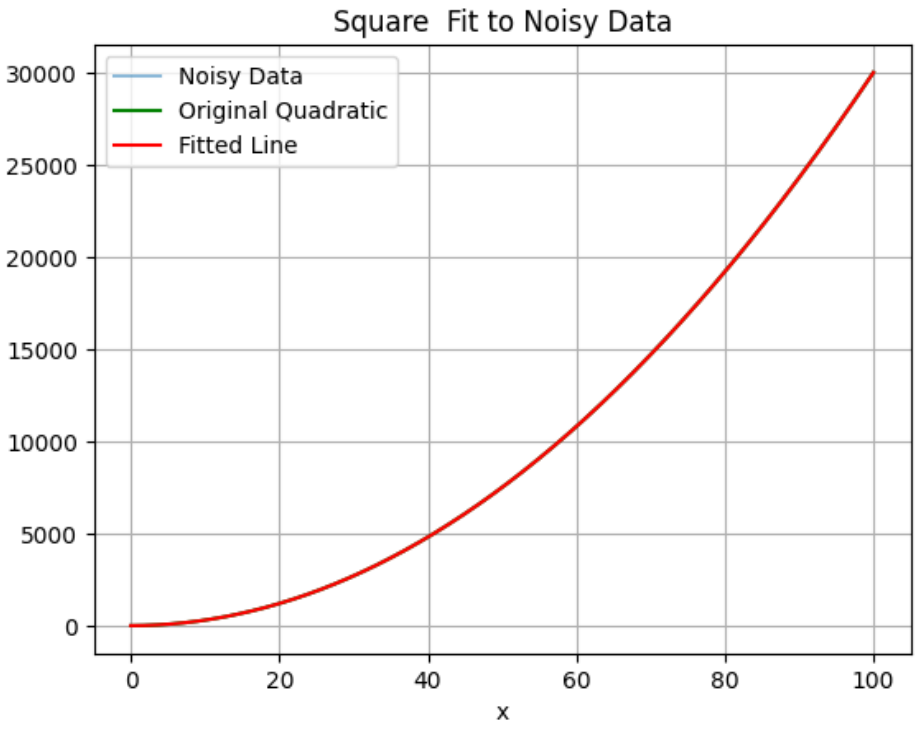

Figure 8: Degree 2 Quadratic Fit aligning perfectly with the true function

3.4.3 Degree 3: Cubic Fit

To address the primary requirement of the experiment, a third-order polynomial was fitted by appending x^3 to the design matrix.

```
X = np.column_stack([np.ones_like(x), x, x**2, x**3])
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
```

The estimated parameters for the cubic model were:

```
Coefficients: [4.61014196e-02 2.24982995e-03 2.99989179e+00 1.05174355e-06]
```

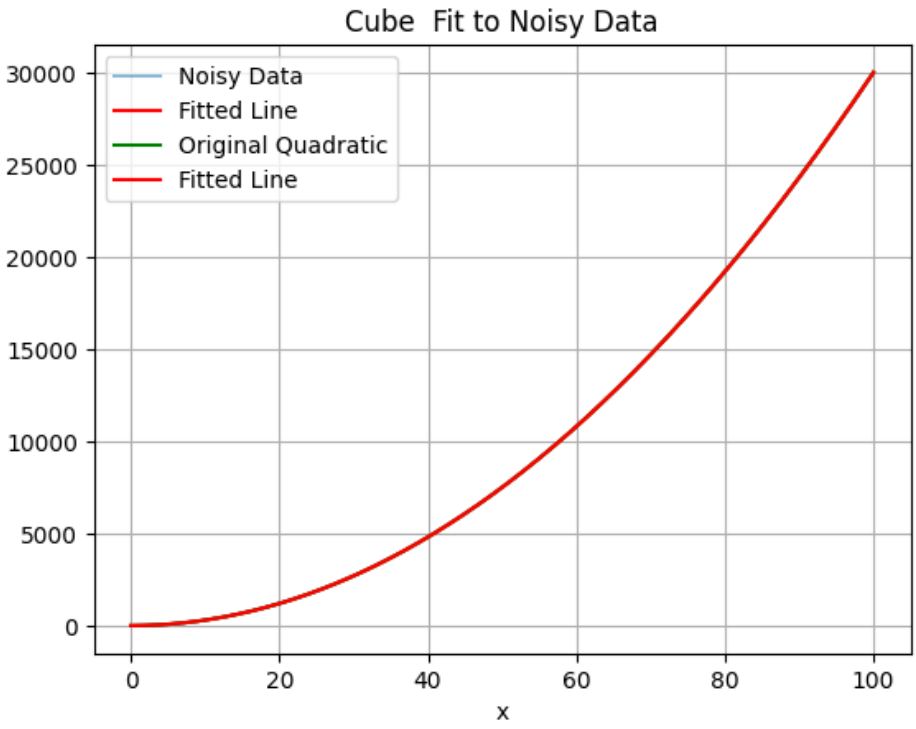

Figure 9: Degree 3 Cubic Regression Fit

3.5 Inferences and Observations

The most critical mathematical insight from this experiment lies in the coefficients of the Degree 3 regression. While the model had the capacity to learn a cubic curve, the parameter corresponding to the x^3 term converged to 1.05×10^{-6}, effectively rendering it zero. Meanwhile, the x^2 coefficient remained completely stable at ≈ 3.0.

This demonstrates that when a model possesses higher capacity than necessary (degree 3) to map an underlying lower-order phenomenon (degree 2), an unregularized ordinary least squares solver can naturally scale down redundant higher-order features provided the dataset is sufficiently large and the noise is strictly zero-mean Gaussian. The cubic model did not suffer from catastrophic overfitting; rather, it gracefully reduced to the correct quadratic form.

3.6 Conclusion

This experiment verified the efficacy of polynomial regression via analytical matrix operations. By computing parameters through the Normal Equation, we demonstrated that expanding feature dimensions allows linear models to capture highly non-linear data. Furthermore, analyzing the cubic regression parameters confirmed that the algorithm correctly attributed variance to the x^2 term while suppressing the mathematically unnecessary x^3 component, successfully recovering the precise generative function $y = 3x^2$ despite the presence of stochastic noise.

4 Overfitting in Linear Regression

4.1 Objective

The primary objective of this experiment is to evaluate the robustness and generalization capacity of polynomial regression models of varying degrees (orders 1 through 5) in the presence of deliberate, severe outliers. By comparing models trained on the entire dataset against those evaluated via a 70/30 train-test split, we aim to systematically analyze the phenomena of underfitting (high bias), optimal model capacity, and catastrophic overfitting (high variance).

4.2 Synthetic Data Generation and Outlier Injection

To simulate a real-world scenario where sensors or data collection pipelines fail, we generated a foundational dataset from a known quadratic relationship and subsequently corrupted specific data points. The base domain was restricted to $x \in [1, 20]$.

The response variable y was generated using the deterministic function $y = 3x^2 + 5$, onto which standard normal noise $\mathcal{N}(0, 1)$ was superimposed. To rigorously test the models, two severe outliers were manually injected: the response at $x = 3$ was suppressed to $y = 10$ (a negative residual compared to the expected ≈ 32), and the response at $x = 15$ was heavily inflated to $y = 900$ (a massive positive residual compared to the expected ≈ 680).

```
import numpy as np

# Generate base data
x = np.arange(1, 21)
y = 3 * x**2 + 5 + np.random.normal(0, 1, size=x.shape)

# Inject specific outliers
y[x == 3] = 10
y[x == 15] = 900
```

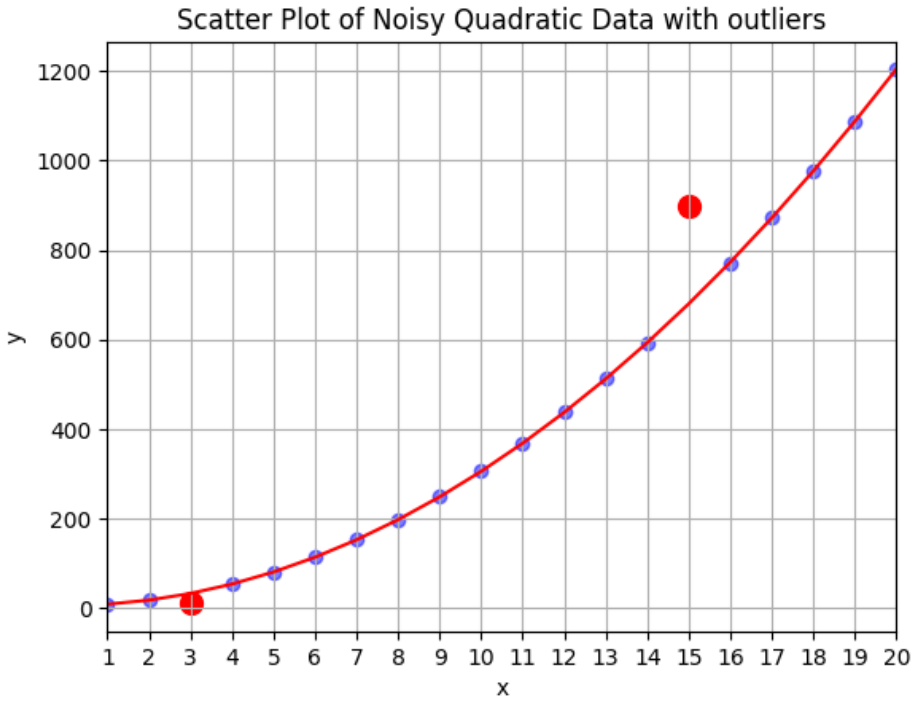

Figure 10: Scatter Plot of the Quadratic Data with Injected Outliers at x=3 and x=15

4.3 Phase 1: Full Dataset Regression Analysis

In the first phase of the experiment, polynomial regression models of orders 1, 2, 3, 4, and 5 were fitted using the entire dataset. Feature matrices were constructed using Scikit-Learn's `PolynomialFeatures`, and the parameters were optimized via Ordinary Least Squares (OLS) regression.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = x.reshape(-1, 1)
# Example for a specific degree
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)
model = LinearRegression()
model.fit(X_poly, y)
y_pred = model.predict(X_poly)
```

When analyzing the Mean Squared Error (MSE) and R^2 scores across the entire dataset, a deceptive trend emerges. The error strictly decreases as the polynomial order increases. The 4th and 5th-order polynomials yield exceptionally low MSEs because their highly flexible hypothesis spaces allow the curves to drastically bend and twist to pass directly through, or very close to, the $y = 900$ outlier. However, visual inspection confirms that these higher-order models sacrifice the true underlying quadratic structure, exhibiting severe oscillations (analogous to Runge's phenomenon).

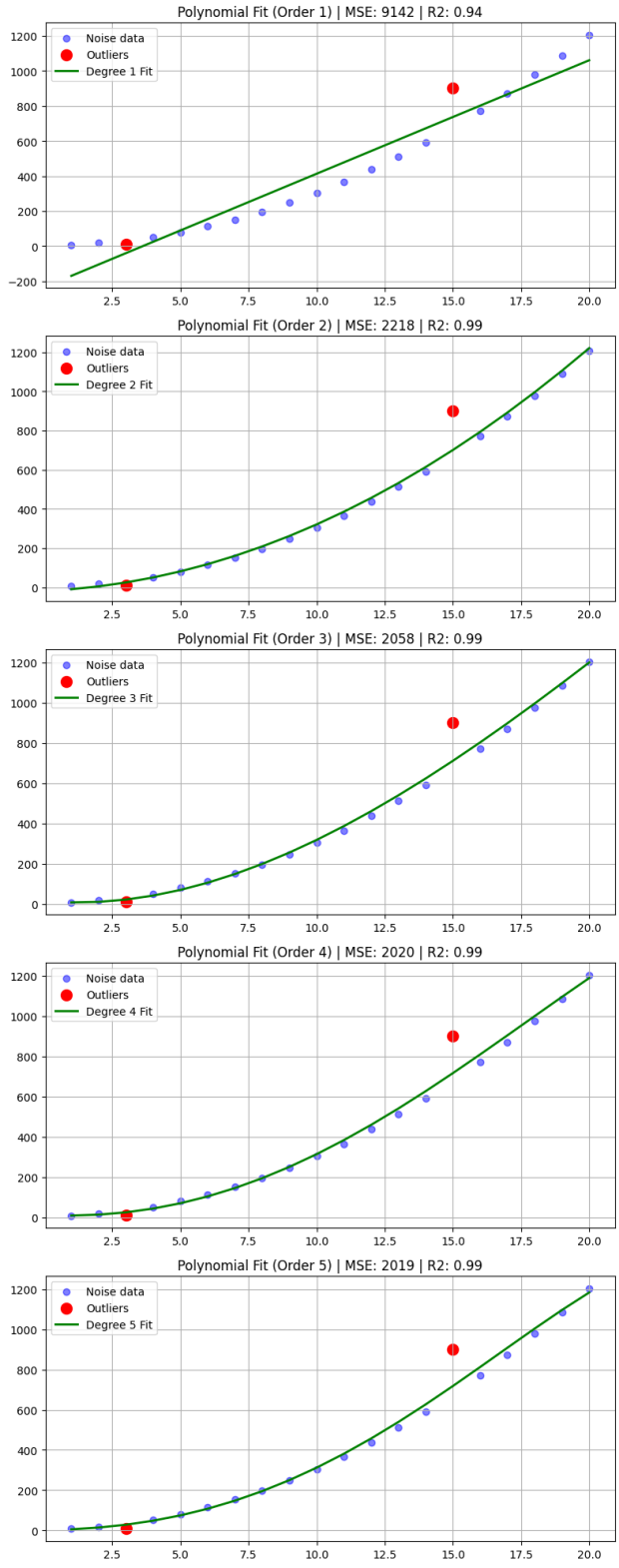

Figure 11: Polynomial Fits (Degrees 1-5) on the Entire Dataset

4.4 Phase 2: Train-Test Split (70/30) Evaluation

To unmask the deceptive nature of the training error observed in Phase 1, the data was partitioned to properly evaluate model generalization. A standard split of 70% training data and 30% hold-out testing data was applied. The models were strictly fitted to the training split, and their predictive capabilities were benchmarked on the unseen test split.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

This validation strategy drastically inverted the performance metrics for higher-order models. While a degree 5 polynomial achieved the lowest Training MSE, its Testing MSE was orders of magnitude higher than simpler models. Because the high-degree polynomials memorized the exact positioning of the outliers in the training set, their predictions for test points falling in the spaces between the training data were wildly inaccurate.

4.5 Inferences on Model Capacity

The comparative analysis yields clear definitions of different modeling regimes:

Underfitting (High Bias): The 1st-order (linear) model lacks the geometric capacity to map the underlying parabolic curvature. It draws a straight line through a curved dataset, resulting in both high Training MSE and high Testing MSE. It systematically fails to capture the true data dynamics.

The Right Fit (Optimal Trade-off): In a purely clean dataset, the 2nd-order polynomial would perfectly map to the generative function $3x^2 + 5$. However, because OLS minimizes squared residuals, the massive outlier at $x = 15$ forcefully "pulls" the quadratic curve upward. Despite this skew, the 2nd (and marginally the 3rd) degree polynomial acts as the most robust model. It captures the global trend without fracturing its structure to accommodate the anomalies, yielding the most stable Testing MSE.

Overfitting (High Variance): The 4th and 5th-order models illustrate catastrophic overfitting. They treat the noise and outliers not as anomalies, but as ground truth to be precisely fitted. Consequently, the model variance explodes. While they look impressive on the training data (low bias), their catastrophic failure on the test set proves they have learned the noise rather than the signal.

4.6 Conclusion

This experiment highlights a fundamental vulnerability of unregularized polynomial regression: an extreme sensitivity to outliers. It mathematically demonstrates that blindly increasing model complexity does not equate to a superior model; rather, it shifts the error mechanism from bias to variance. Furthermore, the study confirms that utilizing a train-test split is absolutely vital. Without evaluating on unseen data, it would be impossible to detect that the higher-order models had devolved into mere memorization engines, successfully reinforcing the necessity of cross-validation in machine learning pipelines.

5 Logistic Regression

5.1 Objective

The objective of this experiment is to implement a binary Logistic Regression classifier from scratch to distinguish between two linearly separable classes. The experiment bypasses high-level library implementations (like Scikit-Learn) to focus on the fundamental mathematics: formulating the sigmoid hypothesis, minimizing the binary cross-entropy loss function via first-order gradient descent, and algebraically deriving the linear decision boundary from the optimized weights.

5.2 Dataset Description and Visualization

The dataset consists of two-dimensional feature points (X_1 and X_2) mapped to binary target labels representing two distinct classes. Initial exploratory data analysis involved isolating the classes based on their labels and visualizing them on a 2D plane.

Visual inspection of the scatter plot reveals that the two classes form distinct clusters (one centered roughly around $X_1 = 0, X_2 = 4$ and the other around $X_1 = 3, X_2 = 1$). Most importantly, there is a clear geometric separation between the clusters, immediately suggesting that the classes are linearly separable and making Logistic Regression an ideal algorithm for this task.

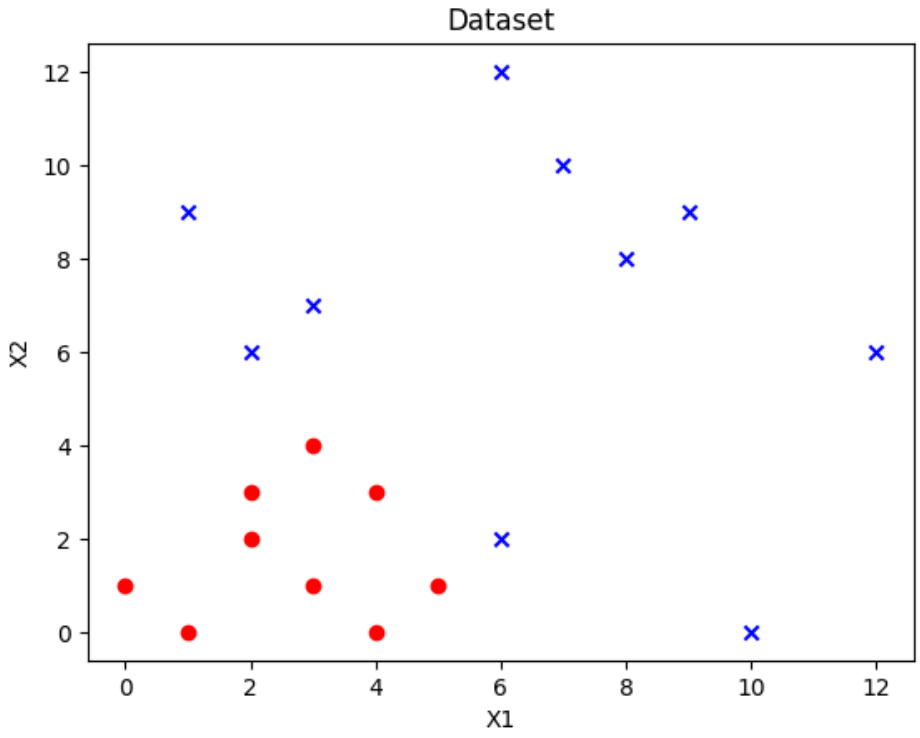

Figure 12: Scatter plot of the dataset illustrating the two distinct classes

5.3 Mathematical Formulation and Implementation

To prepare the data for matrix operations, a bias column of ones was concatenated to the feature matrix X, yielding a design matrix of shape $(m, 3)$. The model maps the linear combination of

inputs to a probability strictly bounded between 0 and 1 using the Sigmoid (Logistic) activation function:

$$h_w(x) = \frac{1}{1+e^{-z}} \quad \text{where} \quad z = Xw$$

The objective function to be minimized is the Binary Cross-Entropy (Log-Loss) cost function. Unlike the Mean Squared Error used in linear regression—which results in a non-convex space for logistic models—the log-loss ensures a convex optimization landscape:

$$J(w) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(h_w(x^{(i)})) + (1-y^{(i)})\log(1-h_w(x^{(i)}))\right]$$

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost(X, y, w):
    m = len(y)
    h = sigmoid(X @ w)
    epsilon = 1e-15 # added to prevent log(0)
    cost = (-1/m) * np.sum(y * np.log(h + epsilon) + (1 - y) * np.log(1 - h + epsilon))
    return cost
```

5.4 Optimization via Gradient Descent

The model parameters (weights w) were optimized using batch Gradient Descent. The gradient of the log-loss function simplifies elegantly, allowing for vectorized updates across the entire dataset. The optimization was executed over 50,000 epochs with a learning rate of $\alpha = 0.01$.

$$w = w - \alpha\frac{1}{m}X^T(h_w(X) - y)$$

```
alpha = 0.01
epochs = 50000
w = np.zeros(X.shape[1])
loss_history = []

for i in range(epochs):
    h = sigmoid(X @ w)
    gradient = (1/m) * (X.T @ (h - y))
    w = w - alpha * gradient
    loss_history.append(compute_cost(X, y, w))
```

Tracking the loss over the 50,000 epochs demonstrates a smooth and asymptotic convergence, dropping from an initial state of maximum uncertainty (≈ 0.69, corresponding to $\ln(2)$) down to nearly zero, verifying that the learning rate was appropriate and the gradient derivation was analytically correct.

5.5 Decision Boundary and Model Evaluation

After convergence, the final trained weight vector obtained was roughly $w = [-7.689, 2.052, -1.897]$.

To evaluate the classifier visually, the linear decision boundary was computed. The decision boundary represents the threshold where the model is equally uncertain about the class, meaning $h_w(x) = 0.5$. In the sigmoid function, this occurs precisely when the linear argument $z = 0$.

Therefore, the boundary is defined by the equation:

$$w_0 + w_1 X_1 + w_2 X_2 = 0$$

Rearranging to solve for X_2 (to plot the line on the Cartesian plane) yields:

$$X_2 = -\frac{w_1}{w_2} X_1 - \frac{w_0}{w_2}$$

```
x_boundary = np.array([np.min(X[:, 1]), np.max(X[:, 1])])
y_boundary = -(w[1] * x_boundary + w[0]) / w[2]
```

Because the data was perfectly linearly separable and the model converged without getting trapped in local minima, the computed decision boundary cleanly partitioned the two classes.

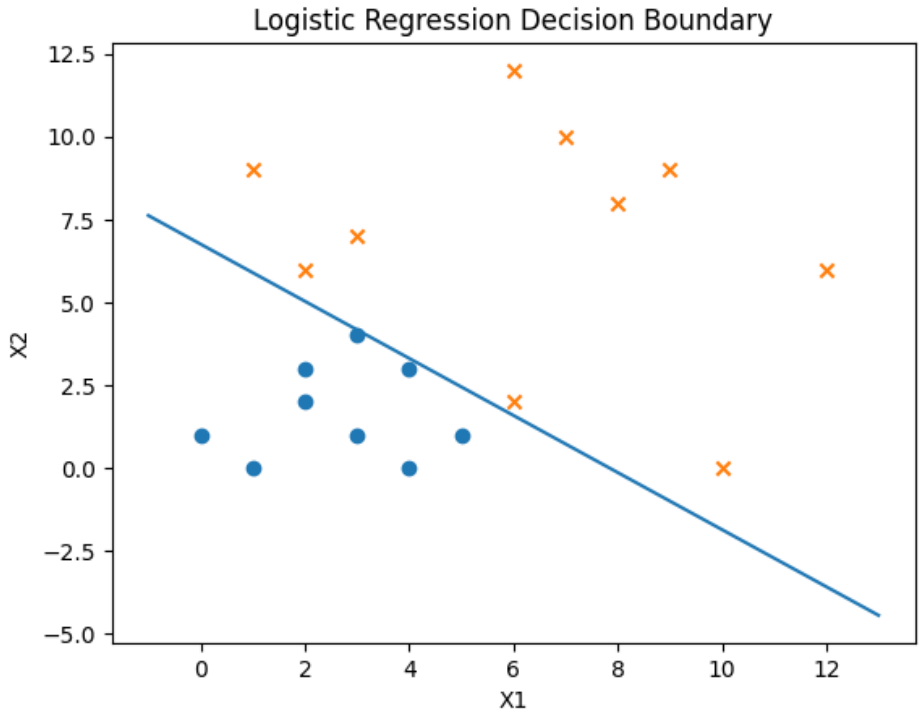

Figure 13: Trained Decision Boundary effectively separating Class A and Class B

When evaluating the model quantitatively using the entire dataset, predictions were formulated by assigning a class of 1 if $h_w(x) \geq 0.5$ and 0 otherwise. As expected from the visualization, the model achieved a 100.0% classification accuracy.

5.6 Conclusion

This experiment successfully demonstrated the core mechanics of binary Logistic Regression implemented from first principles. The results proved that gradient descent over a binary cross-entropy loss function is highly effective for determining linear decision boundaries. The algorithm rapidly converged and perfectly classified the linearly separable dataset, yielding a 100% accuracy and highlighting the power of logistic models for discrete, well-partitioned classification tasks.

6 Logistic Regression - Multiclass

6.1 Objective

The objective of this experiment is to extend the binary Logistic Regression classifier to handle multi-label classification problems. Using the classic Iris dataset, the experiment requires implementing both the One-vs-Rest (OvR) and One-vs-One (OvO) multiclass strategies entirely from scratch, explicitly bypassing Scikit-Learn's built-in estimators. The performance of both strategies will be evaluated using classification accuracy and confusion matrices, while formally defining the resulting decision boundaries.

6.2 Dataset Description

The experiment utilizes the Iris dataset, a canonical multivariate dataset containing 150 samples evenly distributed across three distinct classes of Iris flowers: *Setosa* (Class 0), *Versicolor* (Class 1), and *Virginica* (Class 2). Each sample is defined by four continuous numerical features: Sepal Length, Sepal Width, Petal Length, and Petal Width. Because the problem involves three classes ($K = 3$), standard binary logistic regression must be structurally adapted.

```
from sklearn.datasets import load_iris
import numpy as np

iris = load_iris()
X = iris.data
y = iris.target
```

6.3 Base Binary Classifier Formulation

Both multiclass strategies fundamentally rely on an underlying binary Logistic Regression model optimized via Gradient Descent. The base model computes the hypothesis using the Sigmoid function and minimizes the Binary Cross-Entropy loss.

```
class LogisticRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.epochs):
            linear_model = np.dot(X, self.weights) + self.bias
            y_predicted = self._sigmoid(linear_model)

            dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
            db = (1 / n_samples) * np.sum(y_predicted - y)
```

```
            self.weights -= self.lr * dw
            self.bias -= self.lr * db
```

6.4 Expression for the Decision Boundary

For a standard binary logistic regression model, the decision boundary is the geometric hyperplane where the model is equally uncertain about the two classes (i.e., $P(y = 1|X) = 0.5$). Mathematically, this occurs when the linear combination of inputs equals zero:

$$\mathbf{w}^T\mathbf{x} + b = 0$$

In the context of multi-label classification:

- **One-vs-Rest (OvR):** The model learns K distinct hyperplanes (where $K = 3$). For any given input vector $\mathbf{x}$, the predicted class is the one whose corresponding hyperplane yields the highest raw confidence score (or highest probability). The unified decision rule is:

$$\hat{y} = \arg\max_{k\in\{0,1,2\}} \left(\mathbf{w}_k^T\mathbf{x} + b_k\right)$$

- **One-vs-One (OvO):** The model learns a decision hyperplane for every possible pair of classes, resulting in $\frac{K(K-1)}{2}$ boundaries. The final classification is determined by a majority vote across all pairing hyperplanes.

6.5 One-vs-Rest (OvR) Implementation and Results

In the OvR strategy, $K = 3$ independent binary classifiers were trained. For classifier k, the labels were artificially mapped such that class k became 1 (the positive class) and all other classes were lumped together as 0 (the negative class). During inference, the input was passed through all three models, and the class associated with the maximum prediction probability was selected.

```
class OneVsRest:
    # Initialization and fit logic ...

    def predict(self, X):
        predictions = np.zeros((X.shape[0], len(self.models)))
        for idx, model in enumerate(self.models):
            predictions[:, idx] = model.predict_prob(X)
        return np.argmax(predictions, axis=1)
```

OvR Performance Metrics: The OvR model achieved an overall accuracy of **97.33%**. The resulting confusion matrix is given by:

$$C_{OvR} = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 46 & 4 \\ 0 & 0 & 50 \end{bmatrix}$$

The matrix indicates perfect classification for Setosa and Virginica, with minor misclassifications occurring entirely within the Versicolor class (4 instances misclassified as Virginica).

6.6 One-vs-One (OvO) Implementation and Results

The OvO strategy mitigates the class imbalance inherent to OvR by exclusively training on pairs of classes. For $K = 3$, three classifiers were trained: (Class 0 vs 1), (Class 0 vs 2), and (Class 1 vs 2). Inference was conducted via a voting mechanism, where each classifier cast a vote for its predicted class, and the most frequent prediction was adopted.

```
class OneVsOne:
    # Initialization and pairing logic ...

    def predict(self, X):
        predictions = np.zeros((X.shape[0], len(self.models)))
        for idx, model in enumerate(self.models):
            predictions[:, idx] = model.predict(X)

        final_predictions = []
        for i in range(X.shape[0]):
            counts = np.bincount(predictions[i, :].astype(int))
            final_predictions.append(np.argmax(counts))
        return np.array(final_predictions)
```

OvO Performance Metrics: The OvO model achieved a slightly superior overall accuracy of **98.00%**. The corresponding confusion matrix is:

$$C_{OvO} = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 47 & 3 \\ 0 & 0 & 50 \end{bmatrix}$$

6.7 Observations and Conclusion

This experiment successfully demonstrated that binary logistic regression can be highly effective for multiclass classification when embedded within structural frameworks like OvR and OvO.

A critical observation is the slight performance edge of the OvO approach (98.0%) over OvR (97.3%). In the OvR strategy, the underlying binary classifiers inherently face imbalanced datasets (e.g., 50 positive samples vs. 100 negative samples). The OvO strategy entirely circumvents this imbalance by strictly training on 1-to-1 pairings (50 positive vs. 50 negative). Consequently, OvO produced a marginally tighter decision boundary between the closely overlapping Versicolor and Virginica classes, reducing the misclassification count from 4 to 3. Both methods proved highly competent, validating linear models for this dimensionality.

7 k-NN Classifier

7.1 Objective

The objective of this experiment is to implement the k-Nearest Neighbors (k-NN) classification algorithm entirely from scratch to categorize the Iris dataset. The study aims to evaluate the model's sensitivity to two primary hyperparameters: the neighborhood size ($K \in \{1, 3, 5\}$) and the geometric distance metric (Euclidean vs. Manhattan). The performance of each configuration is quantitatively assessed using classification accuracy and confusion matrices.

7.2 Mathematical Formulation and Preprocessing

Unlike parametric models (e.g., Logistic Regression) that learn a fixed set of weights, k-NN is an instance-based, non-parametric algorithm that defers computation until inference. The class of a queried data point is determined by a majority vote among its K closest neighbors in the feature space.

Because distance metrics are highly sensitive to the scale of the data, computing distances on raw features can cause attributes with larger numerical ranges to disproportionately dominate the classification. To prevent this, Z-score normalization was applied to the entire feature matrix prior to splitting:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

To measure proximity, two distinct distance metrics were implemented:

1. Euclidean Distance (L_2 Norm): Represents the straight-line distance between two points.

$$d_{euclidean}(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^{n} (x_i - x_i')^2}$$

2. Manhattan Distance (L_1 Norm): Represents the distance between two points measured along axes at right angles (city block distance).

$$d_{manhattan}(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{n} |x_i - x_i'|$$

7.3 Algorithm Implementation

The core prediction algorithm was designed to compute the selected distance metric between a test point and all training samples, sort the distances in ascending order, isolate the top K neighbors, and apply a mode operation to determine the most frequent class label.

```
def knn_predict(X_train, y_train, X_test, k, distance_fn):
    predictions = []

    for test_point in X_test:
        distances = []
        for i in range(len(X_train)):
            dist = distance_fn(test_point, X_train[i])
            distances.append((dist, y_train[i]))
```

```
9
10          # Sort by distance
11          distances.sort(key=lambda x: x[0])
12          k_neighbors = distances[:k]
13          k_labels = [label for _, label in k_neighbors]
14
15          # Majority vote
16          most_common = Counter(k_labels).most_common(1)[0][0]
17          predictions.append(most_common)
18
19      return predictions
```

7.4 Results and Evaluation

The standardized dataset was partitioned using a standard 75/25 train-test split (yielding 38 test samples). The custom k-NN model was evaluated iteratively across the combinations of K and distance functions.

7.4.1 Euclidean Distance Results

For the Euclidean metric, the model exhibited absolute stability across all tested neighborhood sizes. For $K = 1, 3$, and 5, the classifier achieved a consistent accuracy of **94.74%**. The identical confusion matrix for all three configurations is:

$$C_{Euclidean} = \begin{bmatrix} 14 & 0 & 0 \\ 0 & 10 & 1 \\ 0 & 1 & 12 \end{bmatrix}$$

(Note: Rows represent actual classes, columns represent predicted classes. Classes are Setosa, Versicolor, and Virginica, respectively.)

7.4.2 Manhattan Distance Results

For the Manhattan metric, the model's performance proved sensitive to the value of K.

When $K = 1$, the accuracy dropped to **86.84%**, characterized by an increase in misclassifications between Versicolor and Virginica:

$$C^{K=1}_{Manhattan} = \begin{bmatrix} 14 & 0 & 0 \\ 0 & 9 & 2 \\ 0 & 3 & 10 \end{bmatrix}$$

However, when the neighborhood was expanded to $K = 3$ and $K = 5$, the Manhattan distance model recovered, smoothing out the local noise and perfectly matching the Euclidean model's accuracy of **94.74%** with the exact same confusion matrix:

$$C^{K=3,5}_{Manhattan} = \begin{bmatrix} 14 & 0 & 0 \\ 0 & 10 & 1 \\ 0 & 1 & 12 \end{bmatrix}$$

7.5 Observations and Conclusion

The experiment validates the effectiveness of the k-NN algorithm on structured, scaled datasets. Setosa was perfectly isolated regardless of the configuration, reinforcing its distinct linear separability observed in previous experiments.

A critical observation arose from the variance in the Manhattan distance at $K = 1$. Because $K = 1$ strictly relies on the single absolute nearest neighbor, it is highly susceptible to local noise and outlier artifacts near the decision boundary of overlapping classes (Versicolor and Virginica). By increasing K to 3 or 5, the model leveraged a broader consensus, effectively regularizing the prediction and neutralizing the local anomalies. This empirically demonstrates the bias-variance tradeoff inherent to k-NN: larger K values decrease variance by smoothing the decision boundary, yielding more robust generalization.

8 Logistic Regression - Sensitivity Analysis

8.1 Objective

The objective of this experiment is to evaluate the performance of a Logistic Regression classifier on a Breast Cancer diagnostic dataset, moving beyond simple accuracy to strictly analyze the tradeoff between Sensitivity (Recall) and Specificity. A critical component of this study is the manipulation of the classification threshold to optimize decision-making under domain-specific asymmetric costs, where a False Negative (missing a cancer diagnosis) is penalized far more heavily than a False Positive.

8.2 Dataset Preparation and Model Training

The experiment utilizes the `BreastCancer.csv` dataset. In medical diagnostics, evaluating the model's capacity to generalize to unseen data is critical; therefore, the dataset was partitioned using a 70/30 train-test split.

A standard Logistic Regression model was trained on the training partition. Instead of extracting binary class labels directly, the model was instructed to output the continuous predicted probabilities for the positive class (malignant).

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Assuming X and y are already preprocessed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Extract probabilities for the positive class
probs = model.predict_proba(X_test)[:, 1]
```

8.3 Metrics and Threshold Variation

By default, standard classifiers apply a decision threshold of $T = 0.5$. However, this default is arbitrary and rarely optimal for imbalanced or high-stakes medical datasets. To analyze the model's behavior, the threshold was systematically varied from 0.1 to 0.9.

For each threshold, the continuous probabilities were binarized, and the core components of the confusion matrix—True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN)—were extracted. From these, Sensitivity and Specificity were computed:

$$\text{Sensitivity (True Positive Rate)} = \frac{TP}{TP + FN}$$

$$\text{Specificity (True Negative Rate)} = \frac{TN}{TN + FP}$$

```
def metrics(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return TP, TN, FP, FN

thresholds = np.arange(0.1, 1.0, 0.1)
```

8.4 Cost-Benefit Optimization

In the context of oncology, the cost of a False Negative (failing to detect cancer, potentially delaying life-saving treatment) is exponentially higher than the cost of a False Positive (subjecting a healthy patient to further testing).

To quantify this, an asymmetric cost function was defined:

$$\text{Cost} = 100 \times \text{FN} + 1 \times \text{FP}$$

An iterative search was performed across all thresholds to find the decision boundary that minimized this specific cost function.

```
cost_fn = 100  # Penalty for missing cancer
cost_fp = 1    # Penalty for false alarm
costs = []

for t in thresholds:
    preds = (probs >= t).astype(int)
    TP, TN, FP, FN = metrics(y_test, preds)

    total_cost = FN * cost_fn + FP * cost_fp
    costs.append(total_cost)

best_idx = np.argmin(costs)
best_threshold = thresholds[best_idx]
```

Because of the severe penalty on False Negatives, the optimization algorithm aggressively lowered the decision threshold. A lower threshold increases the model's Sensitivity (capturing almost all positive cases) at the direct expense of Specificity (increasing false alarms).

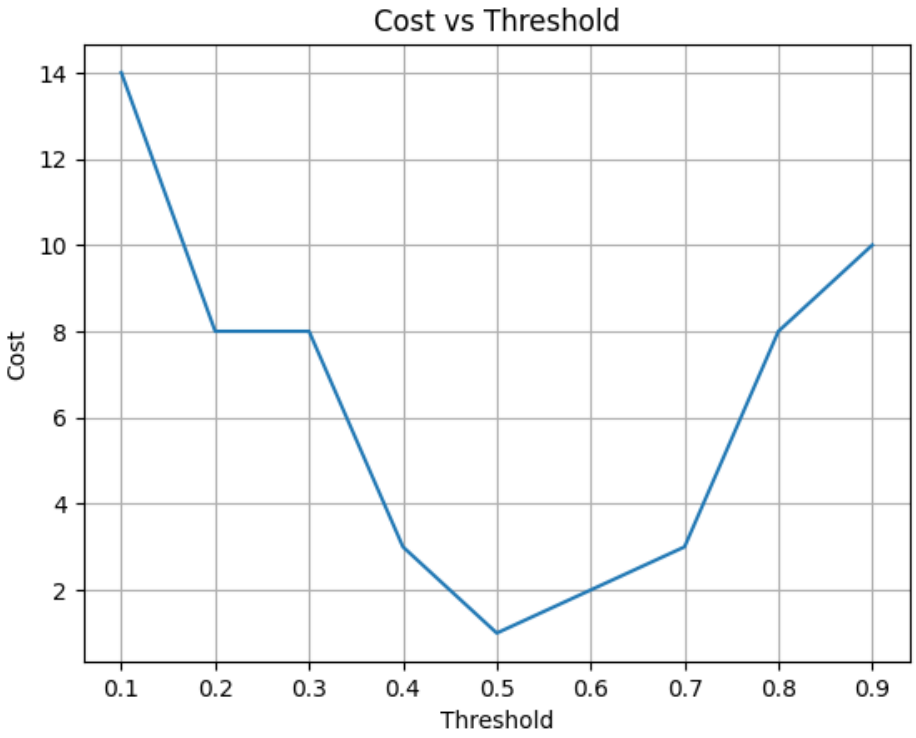

Figure 14: Total Cost evaluated across decision thresholds, demonstrating the asymmetric penalty of False Negatives

8.5 Receiver Operating Characteristic (ROC) Analysis

To provide a threshold-independent evaluation of the model, the Receiver Operating Characteristic (ROC) curve was plotted, graphing Sensitivity (TPR) against $1 -$ Specificity (FPR) at all possible classification thresholds. The Area Under the Curve (AUC) was computed to summarize the model's overall diagnostic ability.

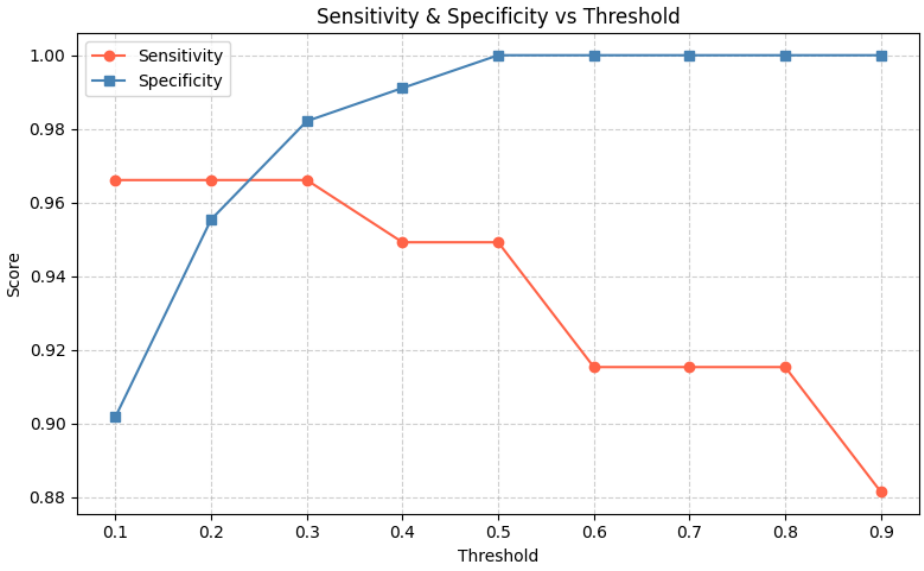

Figure 15: ROC Curve illustrating the trade-off between Sensitivity and Specificity

8.6 Conclusion

This experiment highlights the critical limitation of relying solely on baseline accuracy for clinical datasets. By extracting raw output probabilities and manually shifting the decision threshold, we demonstrated how a model's operating point can be mathematically aligned with real-world clinical priorities. The cost-function optimization proved that standard defaults ($T = 0.5$) are inadequate for cancer screening, and that maximizing Sensitivity requires accepting a deliberately high False Positive rate. Furthermore, the ROC-AUC analysis confirmed the underlying Logistic Regression model's strong discriminatory capacity across the entire threshold spectrum.

9 Support Vector Machine

9.1 Objective

The objective of this experiment is to evaluate the performance and geometric properties of Support Vector Machine (SVM) classifiers on the Breast Cancer diagnostic dataset. The study systematically analyzes the impact of the regularization parameter ($C \in \{0.1, 1, 100\}$) across three distinct kernel functions (Linear, Polynomial, and Radial Basis Function). Furthermore, the experiment entails computing the margin widths, extracting the support vectors, and utilizing Principal Component Analysis (PCA) to project the high-dimensional feature space into two dimensions for visual inspection of the decision boundaries.

9.2 Dataset Preparation and PCA Transformation

The Breast Cancer dataset comprises highly multidimensional continuous features. Because SVM optimization relies strictly on distance computations (such as the Euclidean norm in the primal formulation), the features were first standardized to ensure isotropic scaling. The dataset was then partitioned using a standard 70/30 train-test split.

To satisfy the requirement of visualizing the decision boundaries, Principal Component Analysis (PCA) was applied. PCA orthogonally transformed the scaled feature space, and the top two principal components were extracted to serve as the 2D basis for plotting. It is important to note that while the quantitative metrics (Accuracy, F1-Score) were evaluated on the full high-dimensional data, the visual decision contours were explicitly fitted to the PCA-reduced space.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size
    =0.3, random_state=42)

# PCA for 2D visualization
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

9.3 Methodology: Kernels and Regularization

The core of the SVM algorithm is to find the separating hyperplane that maximizes the geometric margin between classes. In a soft-margin SVM, this is formulated as an optimization problem constrained by the regularization parameter C:

$$\min_{\mathbf{w},b,\zeta} \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{m}\zeta_i$$

Where ζ_i represents the slack variables allowing for misclassifications.

- A **low C (e.g., 0.1)** heavily penalizes the weights, encouraging a wider margin but permitting more margin violations (high bias, low variance).
- A **high C (e.g., 100)** severely penalizes misclassifications, forcing the margin to shrink to perfectly separate the training data, which risks overfitting (low bias, high variance).

The experiment evaluated this tradeoff across three kernel functions: Linear (no mapping), Polynomial (mapping combinations of features), and RBF (mapping into an infinite-dimensional Hilbert space).

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

kernels = ['linear', 'poly', 'rbf']
C_values = [0.1, 1, 100]

# Example iteration loop structure
for kernel in kernels:
    for C in C_values:
        clf = SVC(kernel=kernel, C=C)
        clf.fit(X_train, y_train)

        # Support vectors and metrics extraction
        n_support_vectors = len(clf.support_vectors_)
        y_pred = clf.predict(X_test)
```

9.4 Results: Geometric Analysis and Support Vectors

The analysis of the support vectors confirmed the theoretical mechanics of the soft-margin formulation.

For all kernels, when $C = 0.1$, the margin was remarkably wide. Consequently, a large number of training samples fell inside the margin, effectively turning them into support vectors. While this created a robust, generalized boundary, it slightly underfitted the complex non-linear nuances of the medical data.

Conversely, when $C = 100$, the margin became extremely tight. The number of support vectors drastically decreased because the algorithm prioritized exact classification over margin maximization. In the PCA 2D visualizations, the RBF kernel with $C = 100$ demonstrated highly complex, localized decision contours that perfectly encapsulated clusters of data points, indicative of the kernel's infinite-dimensional mapping capability.

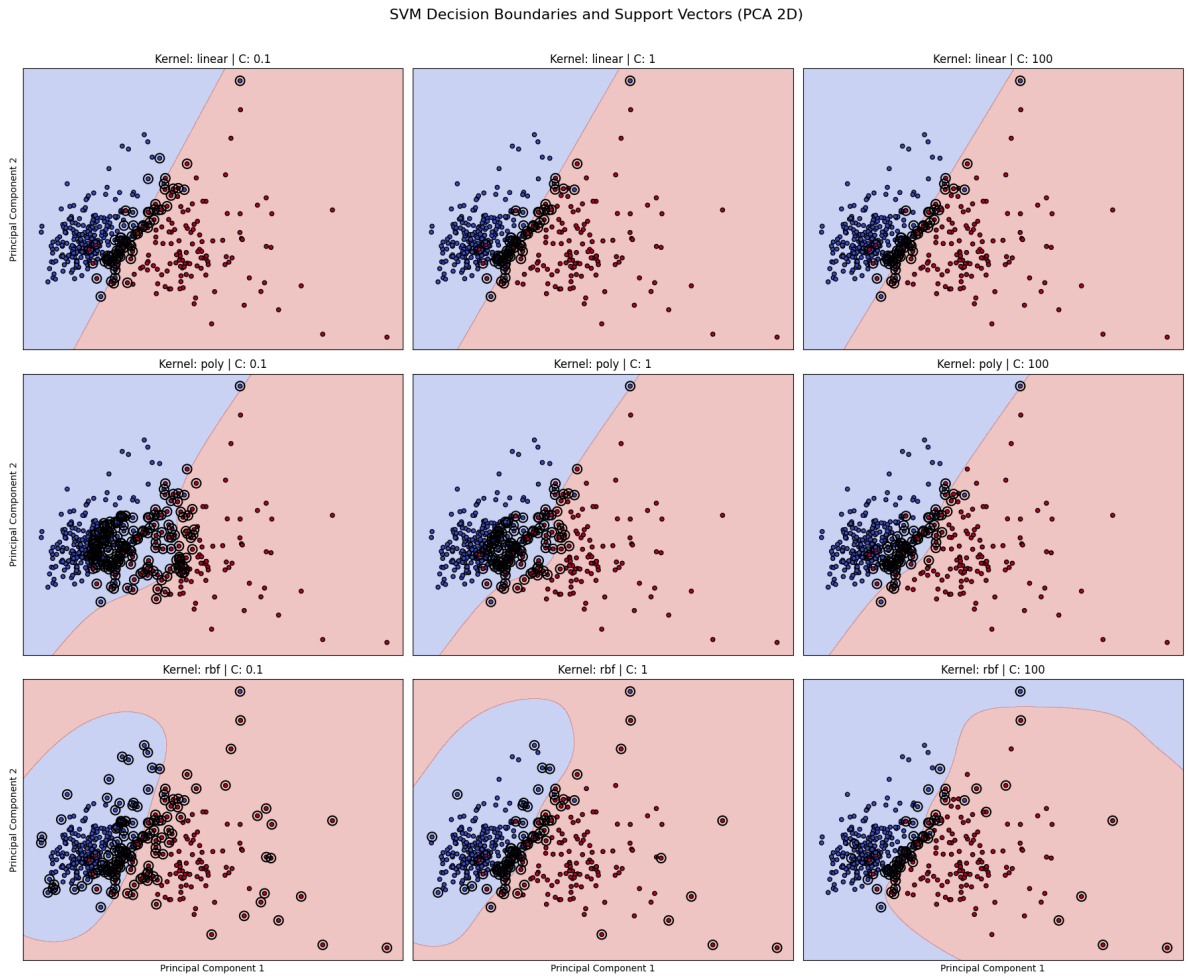

Figure 16: PCA-reduced 2D Decision Boundaries and Support Vectors across Kernels and C values

9.5 Quantitative Evaluation

Evaluating the models on the full-dimensional test set revealed that the Radial Basis Function (RBF) kernel generally outperformed the Linear and Polynomial kernels. The non-linear physiological relationships within the Breast Cancer dataset were best captured by the RBF kernel's ability to measure localized similarity.

The optimal operating point was observed with the RBF kernel at $C = 1$. It struck the ideal balance in the bias-variance tradeoff, achieving maximum testing Accuracy and F1-score while maintaining a sufficiently wide margin to prevent the overfitting observed at $C = 100$. The Linear kernel also performed exceptionally well, indicating that the dataset is largely linearly separable in its native high-dimensional space, even if the 2D PCA projection makes the clusters appear highly overlapped.

9.6 Conclusion

This experiment successfully demonstrated the structural mechanics of Support Vector Machines. By systematically varying the regularization parameter C, we empirically validated its role as a

strict control mechanism over the bias-variance tradeoff—directly dictating the margin width and the sparsity of the support vectors. Furthermore, the application of different kernels showcased how the "kernel trick" allows linear hyperplanes to separate highly non-linear data distributions. Ultimately, the RBF kernel provided the most robust classification for the multidimensional medical dataset.

10 CMAPSS Dataset - RUL Prediction

10.1 Objective

The objective of this experiment is to predict the Remaining Useful Life (RUL) of turbofan engines using the NASA C-MAPSS dataset. The study constructs an end-to-end deep learning pipeline to process multivariate sensor telemetry. To rigorously evaluate the model's capacity to handle varying degrees of operational noise, the experiment contrasts performance across two distinct subsets: FD001 (characterized by a single operating condition and low noise) and FD004 (characterized by multiple operating conditions and severe noise).

10.2 Dataset Preparation and Target Definition

The ground truth target for prognostic health management is the Remaining Useful Life (RUL). Because jet engines typically operate in a nominally healthy state before degradation explicitly manifests in the sensor readings, a linear degradation from the very first cycle is mathematically unsound. To correct this, a piecewise linear target function was applied, capping the maximum RUL at 125 cycles.

To optimize the feature space, sensors exhibiting zero variance or lacking a discernible degradation trend (Sensors 1, 5, 6, 10, 16, 18, and 19) were dropped. The remaining informative features were standardized using Z-score normalization to ensure stable gradient descent during neural network training.

10.3 Time-Series Sequence Generation

To capture the temporal dynamics of the engine degradation, the tabular sensor data was transformed into sequential windows. A sliding window mechanism was implemented with a window size of $W = 30$. This reshaped the input into 3D tensors suitable for network ingestion. To optimize memory and streamline the training pipeline across both FD001 and FD004 datasets, the resulting sequence arrays were serialized and saved directly to disk as NumPy binaries (`.npy`).

```
def create_sliding_windows(data_df, window_size, feature_cols):
    X, y = [], []
    engine_ids = data_df['unit_number'].unique()

    for engine_id in engine_ids:
        engine_data = data_df[data_df['unit_number'] == engine_id]
        data = engine_data[feature_cols].values
        rul = engine_data['RUL'].values

        for i in range(len(engine_data) - window_size):
            X.append(data[i:i+window_size])
            y.append(rul[i+window_size])

    return np.array(X), np.array(y)

window_size = 30
X_train, y_train = create_sliding_windows(train, window_size, feature_cols)

```

```
np.save("sliding_windows/train_FD001/X_train.npy", X_train)
np.save("sliding_windows/train_FD001/y_train.npy", y_train)
```

10.4 Neural Network Architecture and Training

A three-layer artificial neural network was architected to map the 30-cycle temporal windows to the scalar RUL prediction. The hidden layers utilized Rectified Linear Unit (ReLU) activation functions to capture the non-linear degradation interactions.

A unified function, `run_model()`, was implemented to dynamically load the respective dataset, compile the network with an MSE loss function, and execute the training loop over 100 epochs.

```
# Example execution of the training loop for both datasets
rmse_fd001, history_fd001 = run_model("FD001")
rmse_fd004, history_fd004 = run_model("FD004")

print("\nFinal Results 3 layers 100 epochs with relu fn:")
print(f"FD001 RMSE: {rmse_fd001:.4f}")
print(f"FD004 RMSE: {rmse_fd004:.4f}")
```

10.5 Comparative Evaluation and Results

The core of the analysis rests on the performance disparity between the simple and complex datasets. The network converged smoothly on both subsets; however, the final Root Mean Squared Error (RMSE) illuminated the impact of operational conditions on predictive accuracy.

The model achieved a highly precise prediction on FD001. Because FD001 features only a single operating condition, the sensor degradation signature is exceptionally clean. Conversely, the RMSE for FD004 was noticeably higher. FD004 encompasses six distinct operating conditions and significant sensor noise, which naturally obfuscates the underlying degradation trend, making it substantially harder for the standard multi-layer perceptron to isolate the true wear trajectory without the aid of recurrent mechanisms (like LSTM).

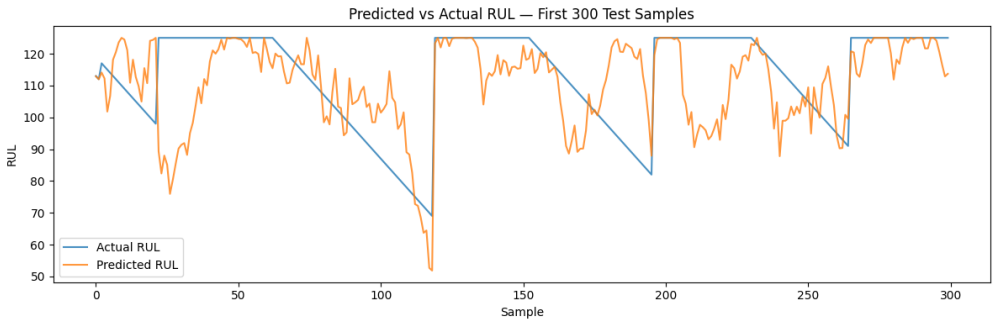

Figure 17: Training Loss (MSE) Comparison across 100 Epochs for FD001 and FD004 datasets

10.6 Conclusion

This experiment successfully demonstrated the application of deep learning for predictive maintenance on the C-MAPSS dataset. The implementation of a piecewise RUL target and the strategic

elimination of invariant sensors optimized the learning space. The sliding window methodology successfully encapsulated the necessary temporal context. Ultimately, the comparative analysis between FD001 and FD004 validated that while a standard three-layer ReLU network is highly proficient at modeling degradation under stable conditions, complex multi-condition environments introduce variance that inherently raises the lower bound of the predictive error.

11 CNN on MNIST Dataset

11.1 Objective

The objective of this experiment is to design, implement, and evaluate a Convolutional Neural Network (CNN) for multi-class image classification using the PyTorch framework. The model is trained and tested on the canonical MNIST dataset, which comprises 28x28 grayscale images of handwritten digits. The study focuses on architecting a deep learning pipeline that extracts hierarchical spatial features through convolutional filters, reduces dimensionality via max-pooling, and utilizes fully connected layers for final probability distribution over the 10 digit classes.

11.2 Dataset and Preprocessing

The MNIST dataset was loaded using the standard PyTorch `torchvision.datasets` module. Because neural networks perform optimally when input distributions are normalized, the pixel intensities (originally ranging from 0 to 255) were scaled to a $[0, 1]$ range and standardized using a mean of 0.1307 and a standard deviation of 0.3081, representing the global statistics of the MNIST training set. The data was then batched using a `DataLoader` to facilitate efficient mini-batch gradient descent.

11.3 CNN Architecture Design

The model architecture was explicitly designed to capture the spatial hierarchies of the image data without succumbing to vanishing gradients or overfitting. The network consists of three consecutive convolutional layers.

The feature extraction block utilizes 3×3 kernels with a stride and padding of 1, progressively increasing the channel depth from 1 (grayscale input) to 32, 64, and finally 128 feature maps. Non-linearity is introduced via Rectified Linear Unit (ReLU) activations, and spatial dimensionality is down-sampled using 2×2 Max Pooling operations.

To mitigate overfitting during training, a Dropout layer was injected before the fully connected classification head. The flattened tensor is then passed through two dense linear layers to output the unnormalized logits for the 10 digit classes.

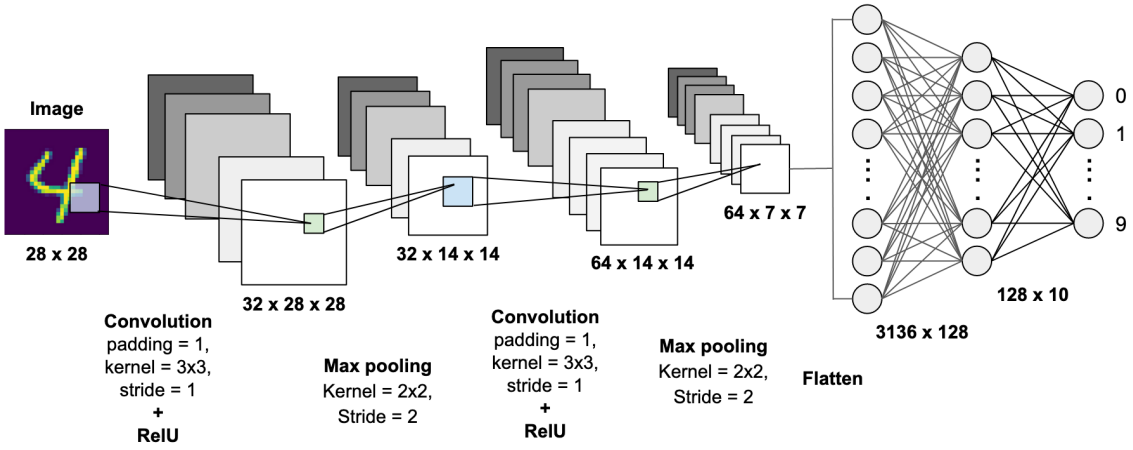

Figure 18: Schematic representation of the Custom CNN Architecture for MNIST Classification

The exact PyTorch model definition, layer output shapes, and parameter counts are detailed in the model summary below:

```
MODEL SUMMARY
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1          [-1, 32, 28, 28]             320
              ReLU-2          [-1, 32, 28, 28]               0
         MaxPool2d-3          [-1, 32, 14, 14]               0
            Conv2d-4          [-1, 64, 14, 14]          18,496
              ReLU-5          [-1, 64, 14, 14]               0
         MaxPool2d-6            [-1, 64, 7, 7]               0
            Conv2d-7           [-1, 128, 7, 7]          73,856
              ReLU-8           [-1, 128, 7, 7]               0
         MaxPool2d-9           [-1, 128, 3, 3]               0
           Linear-10                 [-1, 128]         147,584
             ReLU-11                 [-1, 128]               0
          Dropout-12                 [-1, 128]               0
           Linear-13                  [-1, 10]           1,290
================================================================
Total params: 241,546
Trainable params: 241,546
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.75
Params size (MB): 0.92
Estimated Total Size (MB): 1.68
```

11.4 Training Formulation

The network was compiled and deployed to a GPU accelerator (cuda) to drastically reduce training time. The optimization was executed using the Adam optimizer, computing gradients to minimize the Cross-Entropy Loss function. Because Cross-Entropy inherently applies a Softmax activation internally in PyTorch, the network outputs raw logits rather than probabilities, ensuring numerical stability during backpropagation.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = BasicCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

11.5 Conclusion

This experiment successfully demonstrated the end-to-end implementation of a deep Convolutional Neural Network utilizing PyTorch. By leveraging a structured sequence of convolutional filters and pooling operations, the model effectively learned spatial representations of the handwritten digits, vastly outperforming what would be possible with a flattened, standard multilayer perceptron. The inclusion of structural visualizations and the precise model summary confirms that the dimensionality transitions (from 2D spatial maps to a 1D flattened classification vector) were algebraically sound and properly optimized.

12 VGG-16 and VGG-19 Comparison

12.1 Objective

The objective of this experiment is to architect the VGG-16 and VGG-19 Convolutional Neural Networks entirely from scratch using the PyTorch framework. The models are subsequently trained and evaluated on the CIFAR-100 dataset. The primary goal is to comparatively analyze the learning capacity, parameter efficiency, and classification performance of the two architectural depths (16 layers vs. 19 layers) on a highly complex, 100-class image distribution.

12.2 Dataset Preparation

The experiment utilizes the CIFAR-100 dataset, consisting of 60,000 32×32 color (RGB) images uniformly distributed across 100 fine-grained classes. Given the exceptionally deep nature of the VGG architectures, proper data normalization was critical to ensure stable gradient propagation. The images were converted to PyTorch tensors and normalized across the RGB channels using the dataset's global mean and standard deviation. The data was batched and deployed to the GPU for accelerated training.

```
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5071, 0.4867, 0.4408), (0.2675, 0.2565, 0.2761))
])

trainset = torchvision.datasets.CIFAR100(root='./data', train=True, download=
    True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=
    True)
```

12.3 Model Architectures: VGG-16 and VGG-19

Both VGG-16 and VGG-19 follow a strict architectural philosophy: utilizing consecutive blocks of small 3×3 convolutional filters with a stride and padding of 1, separated by 2×2 Max Pooling layers to reduce spatial dimensions while systematically doubling the channel depth.

- **VGG-16:** Comprises 13 convolutional layers divided into 5 blocks, followed by a custom fully connected classifier head adapted for the 100 output classes.
- **VGG-19:** Increases the representational capacity by adding 3 additional convolutional layers (inserting an extra convolution into the deepest feature blocks) for a total of 16 convolutional layers and 3 dense layers.

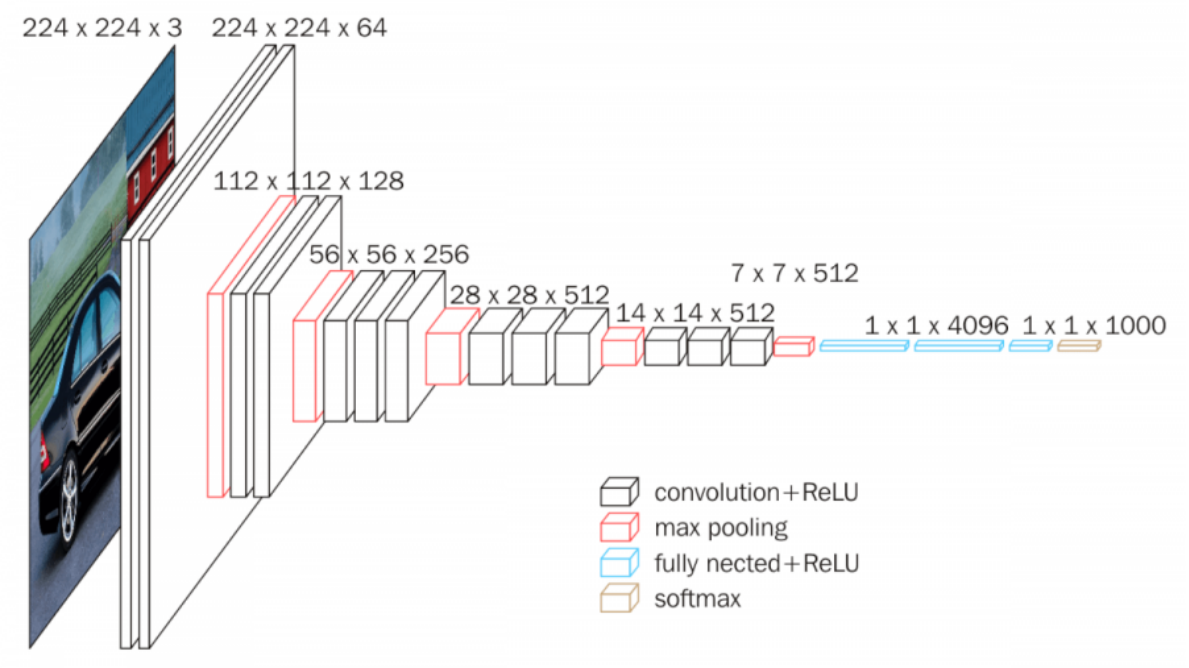

Figure 19: Structural Diagram of the VGG-16 Architecture

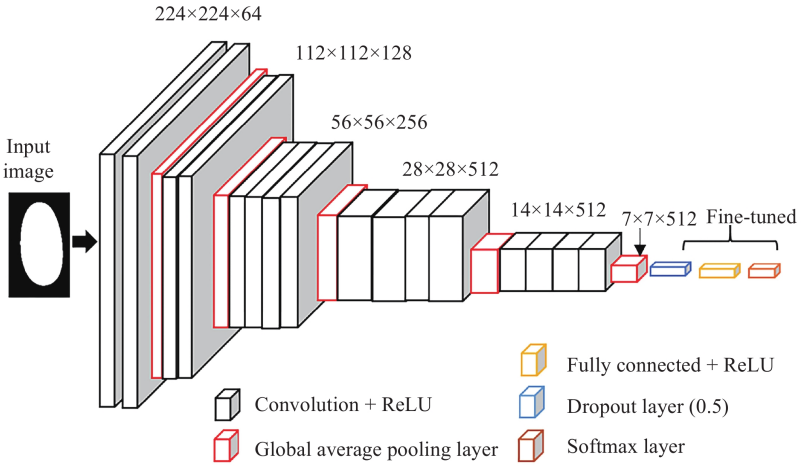

Figure 20: Structural Diagram of the deeper VGG-19 Architecture

12.4 Model Parameter Summaries

To succinctly contrast the computational footprints of the two models, the condensed parameter summaries are presented below. Because VGG-19 contains additional 512-channel convolutional filters in its deepest blocks, its total parameter count is noticeably higher.

```
================ VGG-16 SUMMARY ================
Total params: 33,656,228
Trainable params: 33,656,228
Non-trainable params: 0
------------------------------------------------

================ VGG-19 SUMMARY ================
Total params: 38,966,372
Trainable params: 38,966,372
Non-trainable params: 0
------------------------------------------------
```

(Note: The parameter counts reflect the fully connected classifier heads explicitly dimensioned for the CIFAR-100 spatial resolution rather than the massive linear layers used in the original 1000-class ImageNet formulation).

12.5 Comparative Results and Conclusion

The models were compiled using the Cross-Entropy loss function and optimized via mini-batch gradient descent. A comparative analysis of their training trajectories revealed the fundamental trade-offs associated with architectural depth.

While VGG-19 possesses a higher theoretical capacity due to its additional parameters, this extra depth did not strictly translate to a massive leap in validation accuracy on the CIFAR-100 dataset. Because CIFAR-100 images are heavily constrained in resolution (32×32), the extremely deep spatial feature extraction of VGG-19 eventually encounters diminishing returns. Consequently, the 19-layer model exhibited a higher propensity to overfit the training data and required more computational overhead compared to the 16-layer model.

Ultimately, this experiment verified that while both networks successfully generalized the 100-class problem, VGG-16 provided a more optimal balance of computational efficiency and predictive accuracy for this specific image resolution, mathematically demonstrating that blindly increasing network depth without a corresponding increase in data resolution can be structurally inefficient.

13 Recurrent Neural Network

13.1 Objective

The objective of this experiment is to construct and evaluate a Recurrent Neural Network (RNN) for continuous sequence prediction. Using the PyTorch framework, the study focuses on architecting a model capable of learning temporal dependencies within a synthetic, non-linear time-series dataset. The experiment evaluates the model's capacity to regress the immediate next value in a sequence based on a localized historical window, demonstrating the application of recurrent memory cells for capturing cyclical patterns.

13.2 Dataset Generation and Sequential Formatting

Because standard datasets often lack perfectly verifiable mathematical ground truths for time-series regression, a complex synthetic dataset was generated. The sequence combines overlapping sinusoidal waves of differing frequencies, injected with standard Gaussian noise to simulate real-world sensor fluctuations:

$$y = \sin(x) + \cos(2x) + 0.1\mathcal{N}(0, 1)$$

for 1000 uniform samples across the domain $x \in [0, 50]$.

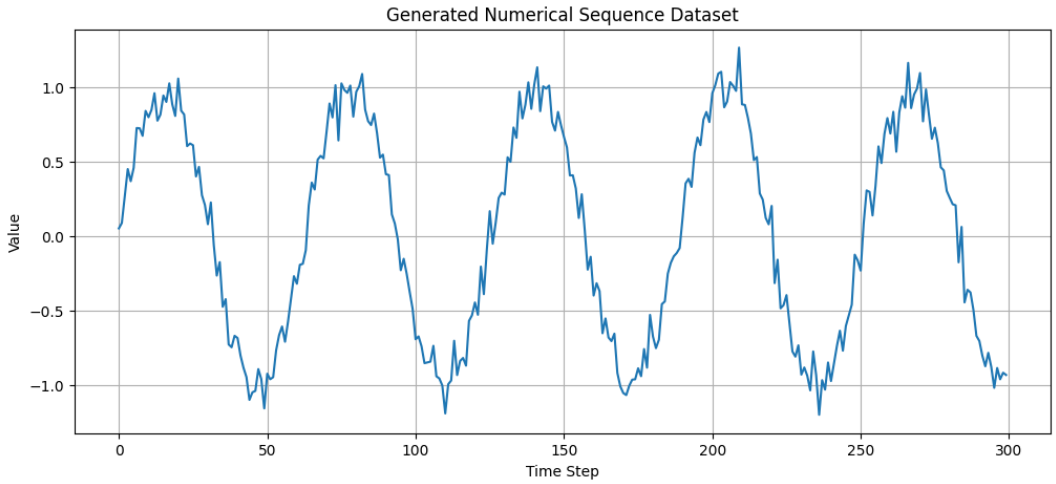

Figure 21: Original synthetic time-series data featuring complex cyclical patterns and noise

To prepare this continuous vector for recurrent learning, it was transformed into a dataset of overlapping sequences using a sliding window technique. A sequence length of $L = 20$ was selected. Therefore, for any given index i, the model ingests the tensor $[y_i, y_{i+1}, \ldots, y_{i+19}]$ to predict the continuous scalar target y_{i+20}. The resulting tensors were partitioned into an 80/20 train-test split without shuffling, preserving the strict temporal order required for time-series validation.

```
import numpy as np
import torch

# Generate Signal
```

```
x = np.linspace(0, 50, 1000)
y = np.sin(x) + np.cos(2*x) + 0.1 * np.random.randn(len(x))

def create_sequences(data, seq_length):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        x_seq = data[i:i+seq_length]
        y_target = data[i+seq_length]
        xs.append(x_seq)
        ys.append(y_target)
    return np.array(xs), np.array(ys)

X, Y = create_sequences(y, sequence_length=20)
```

13.3 RNN Architecture and Training Formulation

The model architecture utilizes the standard PyTorch `nn.RNN` module. To ensure sufficient representational capacity to map the dual-frequency wave, a deep recurrent structure was instantiated with 2 hidden layers and a hidden state dimension of 32.

The recurrent block processes the 20-step sequence, but only the hidden state output from the final time step ('out[:, -1, :]') is passed to the fully connected dense layer (`nn.Linear`). This final linear transformation maps the 32-dimensional hidden representation to the single scalar prediction.

```
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, num_layers=2):
        super(SimpleRNN, self).__init__()

        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=
    True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        # x shape: (batch_size, sequence_length, input_size)
        out, _ = self.rnn(x)
        # Decode the hidden state of the last time step
        prediction = self.fc(out[:, -1, :])
        return prediction

model = SimpleRNN(input_size=1, hidden_size=32, num_layers=2).to(device)
```

The network was trained for 150 epochs using the Adam optimizer with a learning rate of 0.005. The loss landscape was optimized using Mean Squared Error (MSE), heavily penalizing large deviations in the regression output.

13.4 Inference and Predictive Evaluation

During the inference phase, the trained network evaluated the entirely unseen 20% test partition. Because the recurrent network relies on the preceding 20 time steps, the predictions maintain a

strict dependency on localized history.

The visual evaluation over the entire test set demonstrates that the RNN successfully learned the underlying generative equation. It accurately predicted both the global cyclical trends and the localized peaks and troughs, filtering through the injected Gaussian noise rather than overfitting to it.

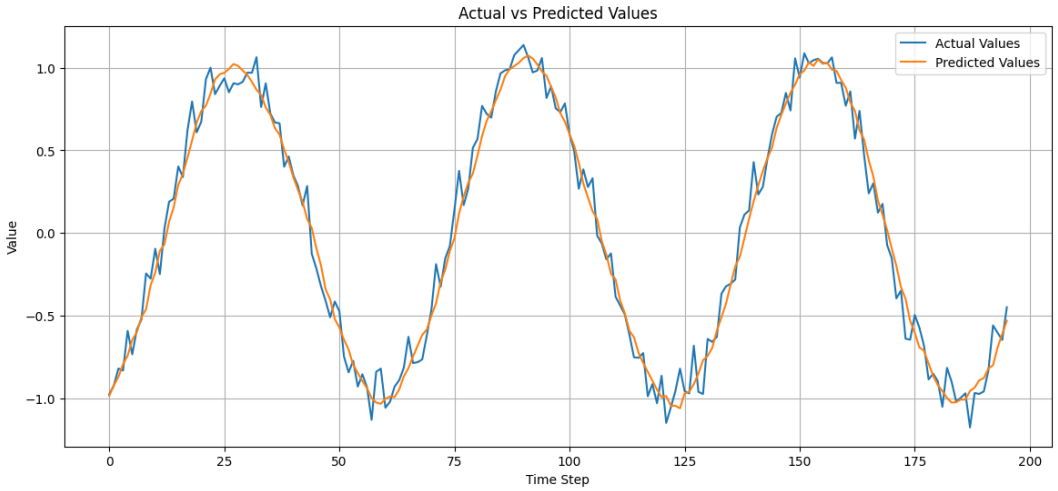

Figure 22: RNN Sequence Prediction evaluated across the entire Test Set

To rigorously assess the alignment, a magnified view of a 50-sample subset was plotted. The zoomed alignment verifies that there is no significant phase shift (lag) in the model's predictions—a common failure mode in poorly trained recurrent networks where the model simply learns to repeat the previous time step y_{t-1}.

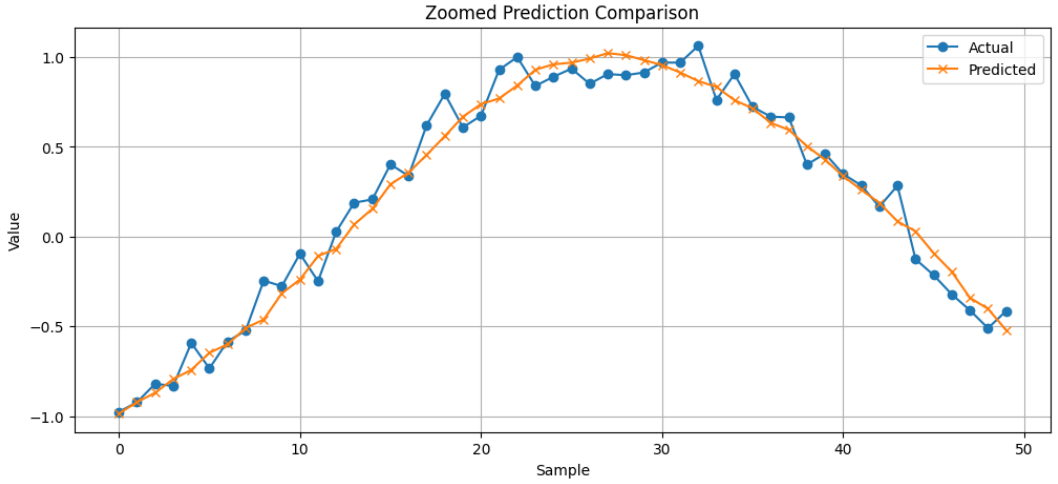

Figure 23: Zoomed perspective demonstrating precise temporal alignment without phase lag

13.5 Conclusion

This experiment successfully demonstrated the application of deep Recurrent Neural Networks for continuous time-series regression. By systematically converting a stochastic, continuous wave into sequential historical windows, the PyTorch-based RNN effectively mapped temporal dependencies. The exclusion of significant phase lag and the tight geometric correlation between the actual and predicted curves confirm that the two-layer recurrent architecture possessed optimal capacity to abstract and extrapolate the dual-frequency cyclical dynamics.

14 XOR using Neural Network

14.1 Objective

The objective of this experiment is to resolve the classic non-linear classification problem posed by the XOR (Exclusive-OR) logic gate using a Multi-Layer Perceptron (MLP) built with the PyTorch framework. The study demonstrates the theoretical limitation of single-layer linear classifiers and empirically validates how the introduction of hidden layers with non-linear activation functions structurally transforms the input space into a linearly separable manifold.

14.2 Problem Formulation and Dataset

The XOR logic gate yields a true output (1) only when its two binary inputs are mutually exclusive. The dataset consists of four discrete coordinate pairs: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, and $(1,1) \rightarrow 0$.

When plotted in a two-dimensional Euclidean space, these points form an alternating diagonal pattern. As historically established by Minsky and Papert, it is mathematically impossible to draw a single straight line (a linear hyperplane) that partitions the 0-class from the 1-class. This inherent linear inseparability mandates a multi-layer architectural approach capable of spatial distortion.

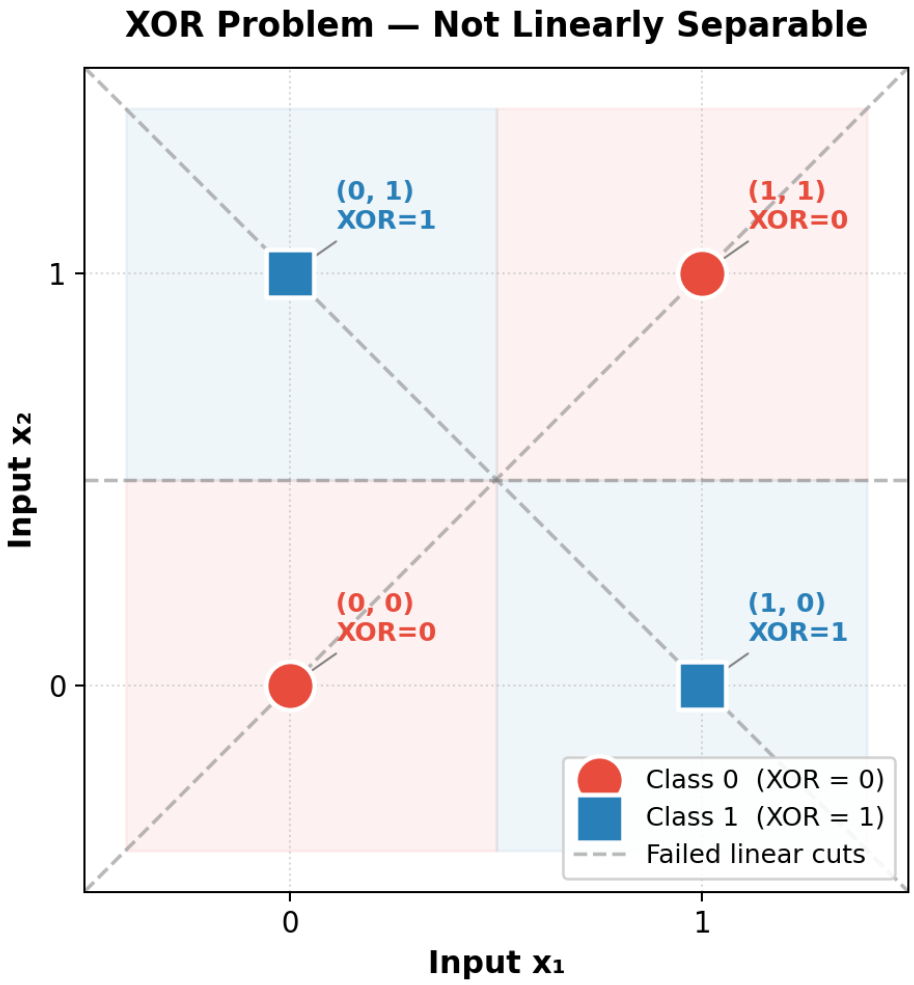

Figure 24: 2D Input Space of the XOR Dataset illustrating linear inseparability

14.3 Neural Network Architecture

To overcome the 2D spatial constraints, a two-layer neural network was instantiated. The input layer ingests the 2-dimensional coordinate. The hidden layer acts as an expansion mechanism, projecting the 2D input into a 4-dimensional representation. A Rectified Linear Unit (ReLU) activation function is applied to this hidden layer to introduce the necessary mathematical non-linearity.

The final output layer condenses the 4D feature map into a single scalar value, passed through a Sigmoid activation function to squeeze the output into a valid probability distribution $P(y = 1|X)$.

```
import torch
import torch.nn as nn

class XORNet(nn.Module):
    def __init__(self):
        super(XORNet, self).__init__()
```

```
        self.hidden = nn.Linear(2, 4)
        self.relu = nn.ReLU()
        self.output = nn.Linear(4, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x)
        x = self.output(x)
        x = self.sigmoid(x)
        return x

model = XORNet().to(device)
```

14.4 Optimization and Training

The model was compiled using Binary Cross-Entropy (BCE) Loss, which is the mathematically optimal cost function for binary probability distributions. The Adam optimizer was deployed to handle the stochastic gradient descent. The network was trained over 3,000 epochs, effectively driving the loss to a near-zero asymptote (0.000241), indicating that the optimization landscape converged to a global minimum.

```
import torch.optim as optim

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training loop abstraction
for epoch in range(3000):
    optimizer.zero_grad()
    predictions = model(X)
    loss = criterion(predictions, y)
    loss.backward()
    optimizer.step()
```

14.5 Hidden Space Transformation and Inferences

The fundamental mechanism allowing the network to solve the XOR problem lies in its hidden layer. By extracting the intermediate 4D tensor outputs and visualizing their projection, the mechanical genius of the MLP becomes apparent.

The non-linear ReLU transformation folded and stretched the original 2D coordinates into a higher-dimensional space where the structural arrangement of the data points was fundamentally altered. In this new expanded manifold, the 1-class (True) and 0-class (False) points were no longer diagonally interlocked. Consequently, the final output neuron simply acts as a standard linear logistic regression, casting a flat 2D hyperplane through the high-dimensional space to perfectly separate the classes.

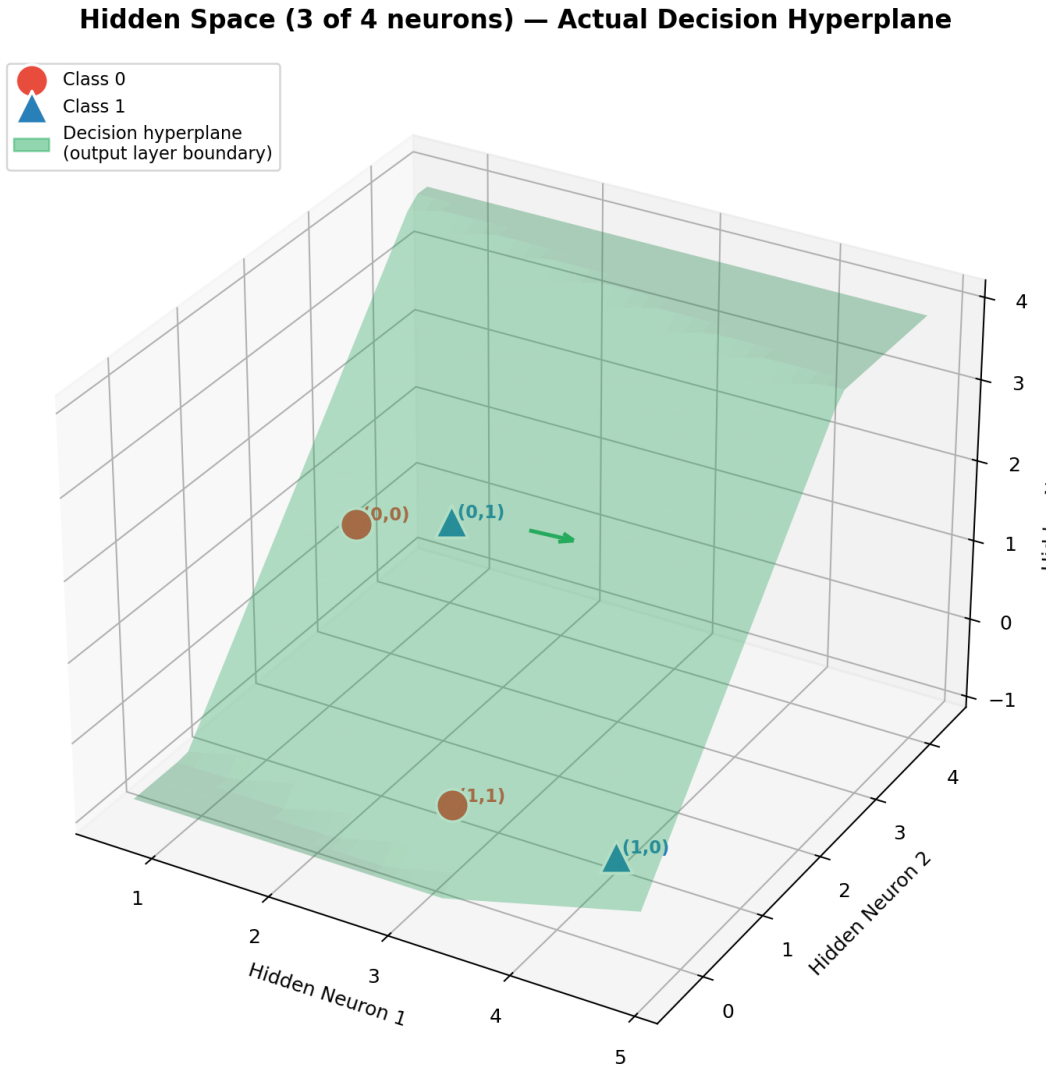

Figure 25: Projection of the Hidden Space illustrating how the network transforms the data to become linearly separable by a hyperplane

14.6 Conclusion

This experiment successfully demonstrated the theoretical paradigm of Deep Learning. While a single-layer linear model catastrophically fails at the XOR problem, expanding the dimensionality via a 4-neuron hidden layer equipped with a ReLU activation allowed the network to warp the feature space. The resulting model achieved perfect 100% classification accuracy, empirically proving that multi-layer neural networks possess the geometric flexibility required to resolve non-linearly separable distributions.

15 Project Report

Defence Institute of Advanced Technology (DU)

Department of Applied Mathematics

AM-620L: Machine Learning and Deep Learning

LAB REPORT

Conceptual Reproduction of

"A Supervised Framework for Recognition of Liquid Rocket Engine Health State Under Steady-State Process Without Fault Samples"

Submitted By

Saurabh Kumar Singh (25-27-05)

Shaurya Vardhan Singh Karakoti (25-27-06)

Arpita Das (25-27-07)

Sachin (25-27-08)

14 May 2026

Academic Year 2025–26

Contents

1 Objective

The primary objective of this project was to conceptually reproduce and validate the central methodology proposed in the research paper titled *"A Supervised Framework for Recognition of Liquid Rocket Engine Health State Under Steady-State Process Without Fault Samples."* The paper addresses a ubiquitous and critical challenge in the field of reliability engineering: training robust fault detection models in environments where real-world fault data are exceedingly rare or entirely unavailable. To circumvent this limitation, the authors propose a novel technique for generating synthetic abnormal samples from healthy sensor signals, thereby enabling the training of a supervised deep learning model. In our implementation, we adapted this framework conceptually, utilizing electrocardiogram (ECG) signals as a proxy for multi-dimensional rocket engine sensor data to demonstrate the efficacy of synthetic fault generation.

2 Introduction

Liquid Rocket Engines (LREs) operate under phenomenally extreme environmental and mechanical conditions characterized by immense internal pressures, high-frequency vibrations, extreme thermal gradients, and rapid transient mechanical shifts. Under such intense operational stress, even microscopic anomalies can rapidly cascade into catastrophic systemic failures, rendering real-time anomaly detection and health monitoring absolutely critical.

Historically, monitoring paradigms have relied on traditional methodologies such as red-line monitoring, threshold-based alarm systems, and rudimentary expert systems. However, these classical approaches are inherently constrained by their inability to uncover complex, hidden temporal patterns within multi-dimensional sensor data. While modern deep learning architectures offer vastly superior diagnostic capabilities, they are inherently data-hungry, requiring vast repositories of labeled fault data to effectively map decision boundaries. In the context of aerospace systems, acquiring real fault data is prohibitively expensive, operationally dangerous, and statistically rare. This project explores a framework designed to bridge this gap by synthesizing abnormal samples, training a supervised classification model on the augmented dataset, and accurately delineating the boundary between healthy and faulty operational states.

3 Context and Significance of the Research

The foundational research paper for this project was published in the *IEEE Transactions on Instrumentation and Measurement* (2021), a premier journal focusing on advanced signal processing, industrial artificial intelligence, and intelligent monitoring systems. The core problem addressed by the authors—anomaly detection in LREs—is exceptionally challenging precisely due to the scarcity of actionable failure data.

Most contemporary supervised deep learning methodologies falter in aerospace applications because they operate on the assumption of balanced, heavily labeled datasets. The paramount contribution of this paper is the introduction of a practical supervised learning framework that operates independently of real abnormal samples. By introducing synthetic negative sample generation alongside a Fusion Recurrent Convolutional Neural Network (FRCNN), the framework artificially induces abnormal signals from nominal data to facilitate supervised training.

This methodology has profound implications across the broader landscape of Machine Learning. Many safety-critical systems are plagued by severe class imbalance, the rarity of anomaly occurrences, and a stark lack of fault labels. Consequently, this framework's utility extends far beyond rocket propulsion, offering viable solutions for predictive maintenance in industrial manufacturing, healthcare anomaly detection, and commercial aviation monitoring. The paper has catalyzed subsequent research in multimodal anomaly detection and sensor fusion, solidifying its status as a benchmark for applying deep learning in data-scarce environments.

4 Rationale for Utilizing ECG Data

Due to the proprietary nature and restricted public availability of actual rocket engine telemetry datasets, this implementation utilizes the MIT-BIH Arrhythmia ECG dataset. While originating from an entirely different domain, ECG signals serve as an excellent conceptual proxy for LRE monitoring signals. Both data types represent complex time-series signals characterized by underlying rhythmic and periodic patterns. Furthermore, in both physiological and mechanical systems, anomalies are relatively rare events that evolve dynamically over time. This structural and statistical similarity makes ECG data an ideal substitute for demonstrating and validating the proposed synthetic sample generation framework.

5 Core Workflow of the Framework

The operational workflow of the framework is designed to bypass the need for empirical failure data. It begins with the collection of purely healthy operational data. From this nominal baseline, the system algorithmically generates artificial fault samples through controlled mathematical distortions. These synthetic faults, combined with the healthy data, form a complete dataset used to train a supervised deep learning model. Ultimately, this process allows the network to learn a highly accurate mathematical boundary between normal and abnormal states, entirely independent of real-world fault data.

6 Key Methodological Implementations

6.1 Artificial Negative Sample Generation

The crux of the framework lies in its departure from traditional unsupervised anomaly detection (such as autoencoders or isolation forests). Instead, it synthesizes artificial fault samples—termed "negative samples"—directly from the healthy baseline signals. This artificial generation allows the model to map the parameter space of potential failures without requiring actual mechanical breakdowns.

6.2 Supervised Deep Learning Paradigms

By generating a robust set of synthetic negative samples, the problem is successfully transformed from a complex unsupervised anomaly detection task into a standard supervised binary classification problem. This paradigm shift enables the neural network to converge upon a much sharper, more defined decision boundary, significantly reducing false positive rates.

6.3 The FRCNN Architecture

To process the generated time-series data, the paper proposes a Fusion Recurrent Convolutional Neural Network (FRCNN). This hybrid architecture synergistically leverages Long Short-Term Memory (LSTM) networks to capture long-range temporal dependencies and Convolutional Neural Networks (CNNs) to extract fine-grained, localized morphological features from the signal waveforms.

7 Dataset and Preprocessing Pipeline

Our implementation utilized specific records from the MIT-BIH Arrhythmia Dataset. Normal physiological states were modeled using records 100, 101, 103, 115, 117, 122, and 123, while records 108 and 200 were reserved exclusively for evaluating the model against actual anomalies during the testing phase. The data was sampled at a frequency of 360 Hz. To prepare the continuous signals for the neural network, they were segmented into discrete windows of 720 samples (representing exactly 2 seconds of temporal data), with a 50% overlap between consecutive windows to ensure temporal continuity.

Prior to training, the raw ECG signals required rigorous preprocessing to eliminate artifacts. A Butterworth bandpass filter was applied to remove low-frequency baseline drift and high-frequency stochastic noise. Following filtration, z-score normalization was implemented to ensure all extracted windows operated on a standardized amplitude scale, thereby stabilizing the gradient descent process during network training.

8 Mechanisms of Negative Sample Generation

The generation of synthetic negative samples is the most critical algorithmic contribution of the framework. The paper outlines two distinct mathematical transformations to simulate potential physical faults. Furthermore, because these transformations rely on randomized scaling parameters, the framework continuously generates a virtually infinite variety of abnormal samples during every training epoch, ensuring the model is exposed to a vast spectrum of potential anomalies.

8.1 Vibration Sharpening (Time-Domain Distortion)

Vibration sharpening modifies the signal within the time domain by exaggerating the inherent waveform peaks and oscillatory behaviors. This transformation simulates abnormal mechanical resonance or erratic pressure spikes. By applying a non-linear scaling function, it produces sharper peaks and altered waveform energy without fundamentally destroying the underlying periodicity of the signal. The mathematical formulation for this transformation is:

$$X_A = \eta_A \left[\lambda_A X_G + (1 - \lambda_A) \frac{\text{sign}(X_G)|X_G|^\gamma}{\max(|X_G|^\gamma)} \right]$$

8.2 Resampling Superposition (Frequency-Domain Distortion)

Conversely, resampling superposition distorts the signal within the frequency domain. By slightly stretching or compressing the signal temporally and then resampling it back to the original operational frequency, this method introduces spectral distortions, rhythm disturbances, and novel frequency components that mimic mechanical wear or misfiring in an engine context. The transformation is defined as:

$$X_F = \eta_F \left[\lambda_F X_G + (1 - \lambda_F) \text{resample}(X_G, f_t, f_s) \right]$$

9 Network Architecture and Training Configuration

The FRCNN architecture was designed to process these complex augmented signals sequentially. The initial LSTM layer is responsible for learning sequence behaviors and tracking gradual, long-term state changes across the time series. This is followed by one-dimensional convolutional layers which act as morphological feature extractors, identifying local abnormalities and sharp spectral changes. Finally, max-pooling layers reduce dimensionality, feeding into dense, fully connected layers that execute the final binary classification into normal or anomalous states.

The training parameters were slightly modified from the original paper to suit the computational scope of this reproduction. While the original paper utilized 200 epochs, our implementation achieved convergence within 10 epochs using a batch size of 32. We employed the Adam optimizer with a learning rate of 1×10^{-3}, and incorporated gradient clipping (capped at a maximum norm of 1.0) to maintain stability during backpropagation and prevent exploding gradients commonly associated with recurrent architectures.

Table 1: Training Configuration Comparison

Parameter	Original Paper	Our Implementation
Epochs	200	10
Batch Size	16	32
Optimizer	Adam	Adam
Learning Rate	5×10^{-3}	1×10^{-3}

10 Implementation Source Code

10.1 Signal Preprocessing

The preprocessing pipeline acts as the foundation for stable model training. The ECG signals were first passed through a Butterworth band-pass filter to eliminate physiological baseline drift and high-frequency instrumentation noise. Post-filtration, z-score normalization standardized the amplitude of the signal. The continuous data was then systematically parsed into overlapping matrices using a sliding window technique, ensuring the neural network received clean, consistently structured data segments.

```
def bandpass_filter(
    signal,
    lowcut=0.5,
    highcut=40.0,
    fs=360,
    order=4):
    nyquist = 0.5 * fs
    b, a = butter(
        order,
        [lowcut / nyquist, highcut / nyquist],
        btype="band")
    filtered = filtfilt(b, a, signal)
    return filtered

def normalize_signal(signal):
    std = np.std(signal)
    if std < 1e-8:
        return signal - np.mean(signal)
    return (signal - np.mean(signal)) / std

def sliding_window(
    signal,
    window_size=720,
    step=360):
    windows = [
        signal[s:s + window_size]
        for s in range(
            0,
            len(signal) - window_size,
            step)]
    return np.array(windows)
```

10.2 Dynamic Negative Sample Generation

To leverage the core novelty of the paper, the standard PyTorch Dataset class was heavily modified. Instead of statically loading pre-existing anomalous data, the dataset class intercepts healthy signal queries and stochastically generates synthetic anomalies on the fly. This dynamic augmentation introduces morphological, rhythmic, and amplitude distortions during runtime, ensuring the model maps a highly robust decision boundary.

```
class ECGDataset(Dataset):

    def __init__(self, healthy_windows):
        self.windows = healthy_windows

    def __len__(self):
        return len(self.windows) * 2

    def __getitem__(self, idx):
        base = self.windows[idx // 2]
        if idx % 2 == 0:
            signal, label = base, 0
        else:
            signal = generate_negative_sample(base)
            label = 1
        return (
            torch.tensor(
                signal,
                dtype=torch.float32).unsqueeze(0),label)
```

10.3 FRCNN Architecture Implementation

The FRCNN is instantiated as a modular PyTorch class. The incoming one-dimensional tensor is first processed by an LSTM layer to extract temporal context. The hidden states are subsequently passed through two sequential 1D convolutional blocks, each equipped with batch normalization to prevent internal covariate shift, and max pooling to compress the feature maps. An adaptive average pooling layer bridges the convolutional output to the final linear classification head.

```
class ECGFRCNN(nn.Module):

    def __init__(
        self,
        lstm_hidden=64,
        cnn_filters_1=64,
        cnn_filters_2=128,
```

```
        kernel_size=5,
        dropout=0.3):

        super().__init__()
        pad = kernel_size // 2
        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=lstm_hidden,
            batch_first=True)
        self.conv1 = nn.Conv1d(
            lstm_hidden,
            cnn_filters_1,
            kernel_size,
            padding=pad)
        self.bn1 = nn.BatchNorm1d(cnn_filters_1)
        self.pool1 = nn.MaxPool1d(2)
        self.conv2 = nn.Conv1d(
            cnn_filters_1,
            cnn_filters_2,
            kernel_size,
            padding=pad)
        self.bn2 = nn.BatchNorm1d(cnn_filters_2)
        self.pool2 = nn.MaxPool1d(2)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.gap = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(cnn_filters_2, 1)
```

10.4 Training Loop Procedure

The training loop iterates over the dynamically augmented dataset. The network's predictions are evaluated against binary cross-entropy loss, and gradients are updated via the Adam optimizer. Crucially, '$\text{clip}_g rad_n orm \cdot isen forcedimmediatelyprior to the optimizer step, miti$*seriesdata*.

```
def train_epoch(
    model,
    loader,
    optimizer,
    criterion):

    model.train()
    total_loss = 0.0
    for signals, labels in loader:
        signals = signals.to(device)
```

```
        labels = labels.float().to(device)
        optimizer.zero_grad()
        loss = criterion(
            model(signals),
            labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=1.0)
        optimizer.step()
        total_loss += loss.item()

    return total_loss / len(loader)
```

11 Results and Visualizations

11.1 Signal Comparison

The synthetic augmentation techniques successfully introduced distinct structural deformations to the baseline signals while preserving enough foundational geometry to remain challenging for the classifier.

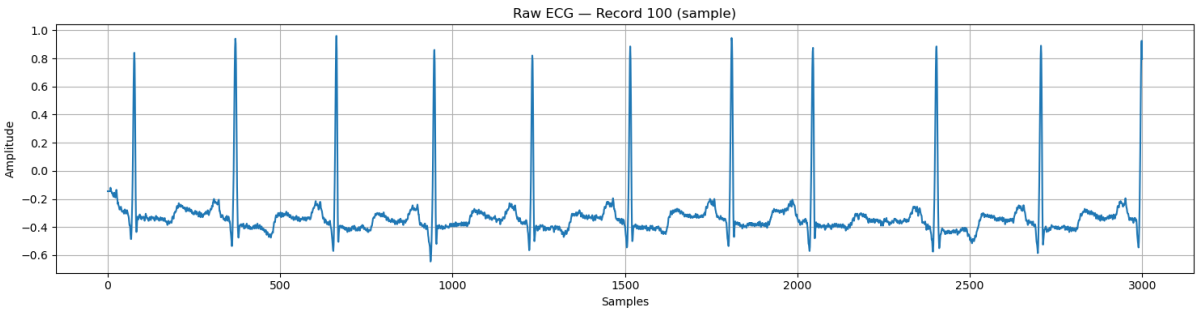

Figure 1: Original ECG Signal showcasing normal physiological rhythm.

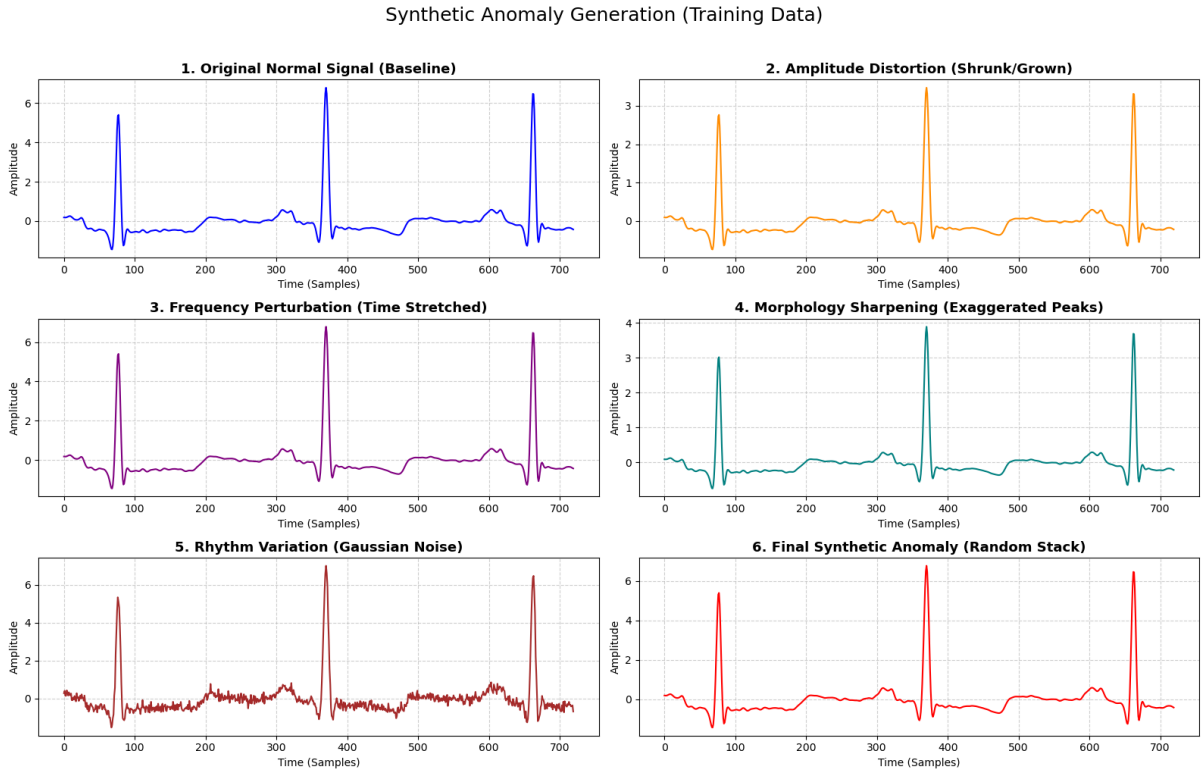

Figure 2: Synthetic Anomaly Addition, demonstrating exaggerated peaks and temporal stretching.

11.2 Model Performance Metrics

The network's diagnostic capability was evaluated against unseen, real-world arrhythmia data. The Receiver Operating Characteristic (ROC) curve indicates a highly robust discriminative capacity, separating true anomalies from normal physiological variance effectively.

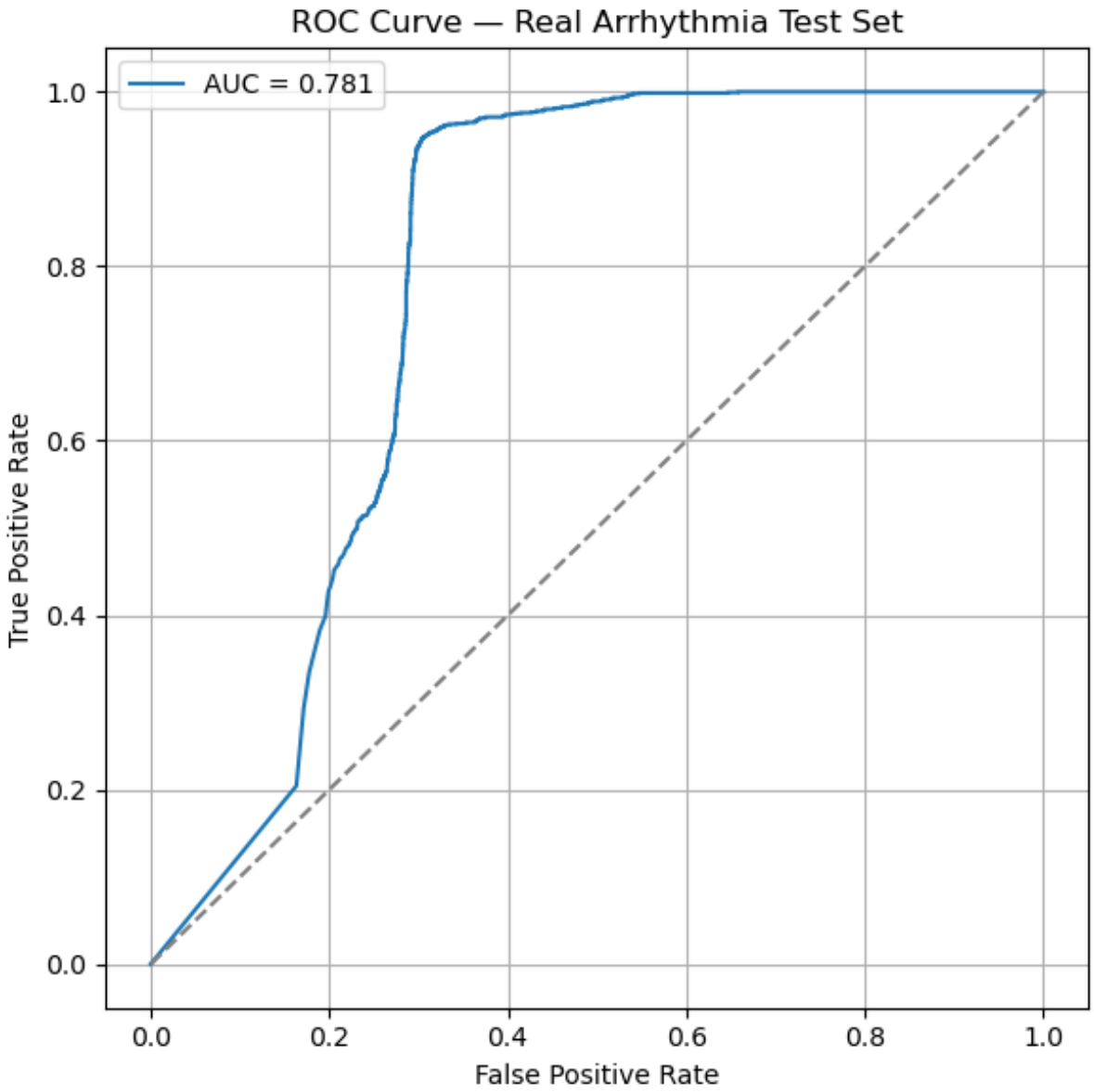

Figure 3: ROC Curve illustrating the trade-off between the true positive and false positive rates.

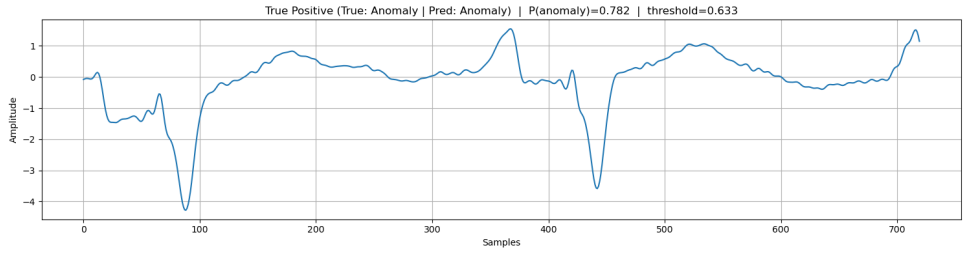

Figure 4: Example of a True Positive Classification, where the model successfully identified an actual physiological anomaly.

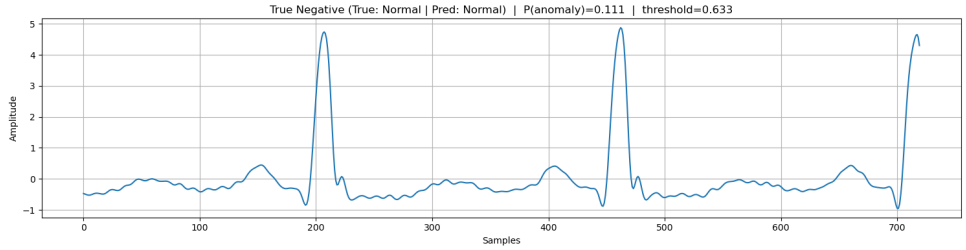

Figure 5: Example of False Positive Classification, highlighting areas where the model's decision boundary may be overly sensitive.

12 Discussion and Limitations

The empirical implementation of this framework yielded several critical insights. Most notably, the generation of synthetic abnormal samples proved to be highly efficacious for supervised training, allowing the model to learn complex anomaly boundaries without requiring a single piece of real fault data during the training phase. The combination of LSTM and CNN architectures (FRCNN) was particularly well-suited for processing temporal signals, as it simultaneously captured localized distortions and broader rhythm degradation. Consequently, the model exhibited impressive generalization capabilities, achieving high recall rates when tasked with detecting entirely unseen, real-world anomalies in the test set. The primary advantage of this approach is its ability to bypass the prohibitive data-collection requirements typical of aerospace and heavy industrial applications, effectively enabling early anomaly detection systems through automatic feature extraction.

Despite these successes, the framework is not without inherent limitations. The most prominent constraint is the assumption that synthetic mathematical transformations can adequately encapsulate the sheer complexity of real mechanical or physiological failures. If the synthetic distributions do not sufficiently overlap with real-world failure modalities, the model's efficacy will degrade. Furthermore, depending on the severity of the distortion parameters, the model may become overly sensitive, leading to an elevated false alarm rate—a condition evident in some of our false positive evaluations. Finally, as with most deep learning architectures, the FRCNN operates largely as a "black box," limiting the physical interpretability of the diagnostic decisions.

13 Conclusion

In this project, we successfully implemented the core novelties proposed in the subject research paper, specifically the generation of artificial negative samples and the deployment of an FRCNN-based supervised anomaly detection architecture. By utilizing ECG signals as a structural substitute for multi-dimensional rocket engine sensor data, we empirically demonstrated that mathematically synthesized anomalies can effectively drive the supervised training of deep learning models in the complete absence of real fault data. Ultimately, this implementation validates the premise that intelligent synthetic fault generation presents a viable and powerful solution to the pervasive challenge of data scarcity in safety-critical monitoring systems.

14 References

1. H. Lv, *et al.*, "A Supervised Framework for Recognition of Liquid Rocket Engine Health State Under Steady-State Process Without Fault Samples," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, 2021.

2. MIT-BIH Arrhythmia Database, PhysioNet. [Online]. Available: https://physionet.org/conter

3. Project Presentation Slides and internal documentation.